

GLIM: 3D Range-Inertial Localization and Mapping with GPU-Accelerated Scan Matching Factors

Kenji Koide, Masashi Yokozuka, Shuji Oishi, and Atsuhiko Banno

Dịch giả: Vũ Tân Hưng

Email: crusadertrask2k@gmail.com

Địa chỉ: RTC Technology, Hà Nội, Việt Nam

Ngày dịch: 29/07/2025

Tóm tắt nội dung—Bài báo này giới thiệu GLIM, một framework định vị và lập bản đồ 3D range-inertial với các yếu tố scan matching được tăng tốc GPU. Module ước lượng odometry của GLIM sử dụng kết hợp giữa fixed-lag smoothing và keyframe-based point cloud matching, cho phép xử lý vài giây dữ liệu range hoàn toàn suy thoái trong khi giảm hiệu quả độ trôi ước lượng quỹ đạo. Nó cũng tích hợp các ràng buộc đặc trưng hình ảnh đa camera theo cách liên kết chặt chẽ để cải thiện thêm tính ổn định và độ chính xác. Module tối ưu hóa quỹ đạo toàn cục trực tiếp giảm thiểu các lỗi đăng ký giữa các submaps trên toàn bộ bản đồ. Cách tiếp cận này cho phép chúng ta ràng buộc chính xác vị trí tương đối giữa các submaps với độ chồng lấp nhỏ. Mặc dù cả hai thuật toán ước lượng odometry và tối ưu hóa quỹ đạo toàn cục đều yêu cầu nhiều tính toán hơn các phương pháp hiện có, chúng ta chứng minh rằng chúng có thể chạy theo thời gian thực nhờ thiết kế cẩn thận của thuật toán đánh giá lỗi đăng ký và toàn bộ hệ thống để tận dụng tối đa xử lý song song GPU.

Index Terms—Tích hợp cảm biến, Định vị và lập bản đồ đồng thời (SLAM).

I. INTRODUCTION

ĐỊNH VỊ VÀ LẬP BẢN ĐỒ ĐỒNG THỜI DỰA TRÊN LIDAR 3D (SLAM) đã được nghiên cứu tích cực do sự xuất hiện của các cảm biến range đa dạng, từ các cảm biến phát hiện và đo khoảng cách bằng ánh sáng (LiDAR) có độ chính xác cao đến các camera độ sâu giá cả phải chăng. Nó đã trở thành không thể thiếu cho nhiều ứng dụng khác nhau, bao gồm giám sát và lái xe tự động.

Hầu hết các hệ thống SLAM dựa trên range hiện đại sử dụng phương pháp ước lượng hai giai đoạn: đầu tiên ước lượng chuyển động ego của cảm biến theo thời gian thực bằng cách sử dụng dữ liệu cảm biến cục bộ (tức là ước lượng odometry) và sau đó sửa các lỗi tích lũy bằng cách xem xét tính nhất quán toàn cục của bản đồ và quỹ đạo.

Đối với ước lượng odometry dựa trên range, sự kết hợp giữa scan-to-model point cloud matching [1] và liên kết chặt chẽ của các ràng buộc range và IMU đã được sử dụng trong nhiều nghiên cứu [2]–[4]. Một thuật toán ước lượng odometry range-IMU đáng chú ý, FAST-LIO2 [5], có độ chính xác tiên tiến với chi phí tính toán hạn chế, kết hợp ràng buộc scan-to-model matching và ràng buộc IMU trên một bộ lọc Kalman lặp hiệu

quả. Tuy nhiên, vì các phương pháp scan-to-model matching và state filtering ngay lập tức cố định ước lượng trạng thái cảm biến hiện tại khi một quan sát mới đến, chúng gặp khó khăn trong việc xử lý các tình huống mà dữ liệu range trở nên hoàn toàn suy thoái trong một khoảnh khắc. Vì không thể xác định trạng thái cảm biến hiện tại chỉ bằng các quan sát quá khứ trong những tình huống như vậy, các phương pháp hiện có dựa trên ước lượng nhân quả không thể tránh được độ trôi ước lượng hoặc sự hỏng hóc.

Tối ưu hóa pose graph là tiêu chuẩn vàng cho tối ưu hóa quỹ đạo toàn cục nhờ hiệu quả và tính ổn định của nó [6]. Dựa trên các quan sát vị trí tương đối giữa các frame thu được bằng scan matching, nó giảm thiểu tổng lỗi vị trí tương đối để sửa độ trôi ước lượng. Trong tối ưu hóa pose graph, mỗi ràng buộc vị trí tương đối được mô hình hóa như một phân phối Gaussian (tức là giá trị trung bình và ma trận hiệp phương sai). Tuy nhiên, trong thực tế, rất khó để ước lượng chính xác ma trận hiệp phương sai của kết quả scan matching do tính phi tuyến và không liên tục của hàm mục tiêu scan matching [7]. Hơn nữa, điểm trung bình (vị trí tương đối) khó có thể thu được chính xác khi các point cloud chỉ có độ chồng lấp nhỏ. Những khó khăn này dẫn đến việc mô hình hóa không chính xác các ràng buộc vị trí tương đối, dẫn đến độ chính xác ước lượng suy giảm cho các quỹ đạo dài với các vòng lặp lớn.

Mục tiêu của chúng tôi trong công trình này là khắc phục những thiếu sót nêu trên trong các hệ thống SLAM dựa trên range hiện có bằng cách giới thiệu một thuật toán đánh giá lỗi đăng ký được tăng tốc GPU cực kỳ nhanh và thay thế các tiêu chuẩn trong cả hai thuật toán ước lượng odometry và tối ưu hóa quỹ đạo toàn cục, cụ thể là scan-to-model matching và tối ưu hóa pose graph với các ràng buộc vị trí tương đối.

Đầu tiên, chúng tôi đề xuất một thuật toán ước lượng odometry kết hợp các ràng buộc lỗi đăng ký point cloud và các ràng buộc IMU cho ước lượng odometry ít trôi với chi phí tính toán có giới hạn. Khác với các phương pháp thông thường, phương pháp được đề xuất liên tục cập nhật các trạng thái cảm biến quá khứ trong cửa sổ tối ưu hóa (tức là fixed-lag smoothing) và biểu diễn mô hình mục tiêu scan matching như một tập hợp các frame quá khứ được chọn (tức là keyframes). Nó lan truyền ước lượng trạng thái cảm biến mới nhất đến các trạng thái quá khứ để sửa độ trôi ước lượng. Cách tiếp cận này làm cho ước lượng odometry cực kỳ mạnh mẽ đối với vài giây dữ liệu range hoàn toàn suy thoái cũng như chuyển động cảm biến nhanh. Hơn nữa,

chúng tôi tích hợp các ràng buộc hình ảnh đa camera vào factor graph ước lượng odometry theo cách liên kết chặt chẽ để cải thiện thêm tính ổn định và độ chính xác.

Sau đó, chúng tôi đề xuất một thuật toán tối ưu hóa quỹ đạo toàn cục trực tiếp giảm thiểu các lỗi đăng ký giữa các submaps trên toàn bộ bản đồ (tức là global multi-scan registration). Khác với tối ưu hóa pose graph, phương pháp này tránh xấp xỉ Gaussian của các ràng buộc vị trí tương đối và trực tiếp tính toán các lỗi đăng ký point cloud trên factor graph. Nó cho phép vị trí tương đối giữa các point cloud với độ chồng lấp rất nhỏ được ràng buộc chính xác. Hơn nữa, chúng tôi tích hợp các ràng buộc IMU vào factor graph tối ưu hóa toàn cục theo cách liên kết chặt chẽ để ổn định hóa tốt hơn việc tối ưu hóa toàn cục và giảm lỗi ước lượng quỹ đạo trong bốn bậc tự do (DoFs).

Cả hai thuật toán được đề xuất đều yêu cầu nhiều tính toán hơn so với các thuật toán thông thường. Do đó, việc chạy chúng theo thời gian thực được coi là không khả thi. Trong công trình này, bằng cách thiết kế cẩn thận thuật toán đánh giá lỗi đăng ký và toàn bộ framework để tận dụng tối đa xử lý song song GPU, chúng tôi chứng minh rằng các thuật toán này có thể chạy theo thời gian thực trên một GPU cấp tiêu dùng đơn lẻ.

Đóng góp: Các đóng góp chính của công trình này như sau:

- Chúng tôi mở rộng yếu tố lỗi đăng ký GICP voxelized được tăng tốc GPU [8] với xác thực tương ứng dựa trên định hướng bề mặt và voxelmap đa độ phân giải để cải thiện tính ổn định và độ chính xác trong môi trường trong nhà. Chúng tôi cũng đề xuất một cơ chế tuyến tính hóa hiệu quả cho các yếu tố dựa trên GPU để giảm thiểu overhead đồng bộ hóa CPU-GPU và tối đa hóa tốc độ tối ưu hóa.
- Chúng tôi đề xuất một thuật toán ước lượng odometry range-IMU liên kết chặt chẽ dựa trên keyframe cực kỳ mạnh mẽ đối với chuyển động cảm biến nhanh và sự suy thoái tạm thời của dữ liệu range. Chúng tôi trình bày một phần mở rộng visual-range-IMU đa camera để cải thiện thêm độ chính xác ước lượng.
- Chúng tôi đề xuất một thuật toán tối ưu hóa quỹ đạo toàn cục dựa trên giảm thiểu lỗi đăng ký toàn cục với các ràng buộc IMU liên kết chặt chẽ. Chúng tôi giới thiệu khái niệm *endpoints* của submap để ràng buộc mạnh mẽ các vị trí submap với khoảng thời gian lớn bằng các ràng buộc IMU.
- Framework được đề xuất được thiết kế cẩn thận để tổng quát với mô hình cảm biến. Chúng tôi chứng minh rằng nó có thể xử lý các cảm biến range-IMU đa dạng bao gồm không chỉ LiDAR mà còn camera độ sâu và stereo.
- Chúng tôi phát hành phần lớn mã của framework được đề xuất, GLIM, như mã nguồn mở¹.

II. RELATED WORK

Các framework SLAM hiện đại bao gồm hai module: 1) một module ước lượng odometry ước lượng chuyển động ego của cảm biến theo thời gian thực bằng cách sử dụng dữ liệu cảm biến cục bộ và 2) một module tối ưu hóa quỹ đạo toàn cục sửa các lỗi ước lượng tích lũy bằng cách xem xét tính nhất quán toàn cục. Chúng tôi tóm tắt các xu hướng gần đây trong cả ước lượng odometry và tối ưu hóa quỹ đạo toàn cục cho SLAM dựa trên

range và giải thích khái niệm của framework được đề xuất trong phần này.

A. Odometry Estimation

Feature vs. direct point cloud registration: Đăng ký point cloud là một yếu tố quan trọng cho ước lượng chuyển động ego của cảm biến. Hai loại phương pháp đăng ký point cloud thường được sử dụng cho SLAM dựa trên range, cụ thể là các phương pháp dựa trên đặc trưng, trích xuất các điểm mặt phẳng và cạnh và căn chỉnh point cloud bằng cách khớp các điểm này, và các phương pháp trực tiếp, tránh trích xuất đặc trưng và sử dụng hầu hết các điểm đầu vào để căn chỉnh point cloud.

Đăng ký point cloud dựa trên đặc trưng lần đầu được đề xuất cho LOAM [1]. Nó trích xuất các điểm mặt phẳng và cạnh dựa trên độ mượt cục bộ, và sau đó căn chỉnh point cloud bằng cách giảm thiểu khoảng cách khớp point-to-edge và point-to-plane. Thường thì, một quá trình gọi là frame-to-model matching theo sau scan matching frame-by-frame để giảm độ trôi ước lượng. Trong frame-to-model matching, các point cloud đã căn chỉnh được tích lũy trong một target point cloud đơn lẻ (tức là map model) và vị trí cảm biến được ước lượng bằng cách căn chỉnh scan hiện tại với model point cloud. Nhiều công trình sử dụng kết hợp đăng ký dựa trên đặc trưng và frame-to-model matching [9]–[11] do hiệu quả của nó. Cách tiếp cận này yêu cầu điều chỉnh cẩn thận việc trích xuất đặc trưng, và các thuật toán trích xuất đặc trưng dành riêng cho các cảm biến cụ thể thường được sử dụng [3], [9], [10].

Đăng ký point cloud trực tiếp tránh trích xuất đặc trưng và sử dụng hầu hết các điểm để thực hiện scan matching [5], [12]–[15]. Các biến thể của thuật toán ICP với mô hình bề mặt cục bộ (ví dụ: point-to-normal ICP [12] và generalized ICP [14], [16]) thường được sử dụng do độ chính xác và tính mạnh mẽ của chúng. Mặc dù chi phí tính toán của các phương pháp đăng ký trực tiếp cao hơn so với các phương pháp dựa trên đặc trưng, nhưng chúng mạnh mẽ hơn đối với các thay đổi cảm biến và môi trường vì chúng không yêu cầu quá trình trích xuất đặc trưng tinh tế. Phương pháp scan-to-model matching cũng thường được sử dụng cho các phương pháp đăng ký trực tiếp [5], [12], [14], [15].

Loose vs. tight IMU coupling: Với sự tiến bộ trong SLAM visual-inertial [17]–[19], tích hợp IMU đã trở thành một kỹ thuật quan trọng cho SLAM dựa trên range [10]. Việc sử dụng IMU cho phép dự đoán chuyển động cảm biến ở tần số 100 đến 1000 Hz, tạo điều kiện cho các ước lượng ban đầu tốt về vị trí cảm biến và sửa lỗi biến dạng của point cloud dưới chuyển động cảm biến nhanh. Hơn nữa, các phép đo IMU cung cấp thông tin về hướng trọng lực, cho phép giảm độ trôi ước lượng quỹ đạo trong bốn DoFs [17].

Các phép đo IMU và point cloud có thể được tích hợp bằng cách sử dụng loose coupling scheme, xem xét riêng biệt ước lượng dựa trên range và ước lượng dựa trên IMU và tích hợp các kết quả ước lượng trong không gian pose bằng một phương pháp như extended Kalman filter [20] hoặc factor graph optimization [10], [21]. Vì cách tiếp cận này cho phép dữ liệu cảm biến không đồng nhất (ví dụ: LiDAR, camera, và IMU) được tích hợp dễ dàng, một số phương pháp sử dụng IMU-centric loose coupling approach để ước lượng mạnh mẽ trạng thái cảm biến trong môi

¹<https://github.com/koide3/glim>

trường khắc nghiệt (ví dụ: mỏ ngầm) [22]–[24]. Mặc dù loose coupling scheme hiệu quả về mặt tính toán và có thể dễ dàng xử lý dữ liệu cảm biến không đồng nhất, nhưng khó để lan truyền và tích hợp chính xác sự không chắc chắn cho mỗi cảm biến với loose coupling scheme, và tight coupling schemes chính xác và mạnh mẽ hơn loose coupling schemes [2], [17].

Một tight coupling scheme ước lượng các trạng thái cảm biến bằng cách trực tiếp giảm thiểu một hàm mục tiêu thống nhất kết hợp các ràng buộc dựa trên range và IMU. Cách tiếp cận này cho phép ước lượng mạnh mẽ các trạng thái cảm biến trong môi trường mà thông tin hình học đủ không có sẵn thông qua dữ liệu range vì các ràng buộc IMU giúp ràng buộc trạng thái cảm biến dựa trên thông tin quán tính. Do độ chính xác và tính mạnh mẽ của chúng, các phương pháp LiDAR-IMU liên kết chặt chẽ đã được nghiên cứu rộng rãi trong những năm gần đây [2], [3], [25], [26].

Filtering vs. optimization: Tight coupling schemes có thể được phân loại thành các phương pháp dựa trên filtering và optimization. Phương pháp filtering chỉ giữ trạng thái cảm biến mới nhất và ngay lập tức loại bỏ các trạng thái cũ khi một quan sát mới đến. Phương pháp optimization giữ các frame quá khứ hoạt động và tối ưu hóa chúng thông qua tối ưu hóa phi tuyến. Để giới hạn chi phí tính toán, chỉ một tập con của các frame quá khứ (ví dụ: một số keyframes hoặc frames trong sliding window) thường được xem xét cho tối ưu hóa. Mặc dù phương pháp filtering hiệu quả vì nó chỉ giữ và tối ưu hóa trạng thái cảm biến mới nhất, nhưng nó nhanh chóng tích lũy các lỗi tuyến tính hóa. Trong bối cảnh visual SLAM, đã được chứng minh rằng phương pháp dựa trên optimization vượt trội hơn phương pháp dựa trên filtering về độ chính xác vì nó tái tuyến tính hóa các phép đo quá khứ [27], [28].

Mặc dù có ưu thế lý thuyết của phương pháp dựa trên optimization, trong bối cảnh SLAM dựa trên range, nhiều nghiên cứu đã sử dụng phương pháp filtering cho IMU fusion [3], [5]. Điều này có thể được thực hiện vì hai lý do. Thứ nhất, point cloud matching tốn kém hơn về mặt tính toán so với visual feature matching và việc xem xét chung nhiều frame trong vài giây của optimization window theo thời gian thực được coi là không khả thi. Thứ hai, phương pháp frame-to-model matching được sử dụng rộng rãi cần ngay lập tức cố định ước lượng vị trí cảm biến để tích lũy các điểm vào target model point cloud. Cập nhật các ước lượng trạng thái cảm biến của các frame quá khứ không tạo ra sự khác biệt đáng kể vì model point cloud đã được đóng băng.

Continuous-time pose representation: LiDAR thường sử dụng cơ chế quét laser cho các phép đo trường nhìn rộng có thể gây ra biến dạng point cloud khi cảm biến đang chuyển động. Để bù đắp cho sự biến dạng này, một số công trình đã sử dụng biểu diễn vị trí cảm biến liên tục dựa trên nội suy vị trí, cho phép thu được vị trí cảm biến tại bất kỳ bước thời gian nào [11], [29]–[31]. So với biểu diễn vị trí rời rạc theo thời gian, biểu diễn liên tục theo thời gian cho phép thu được vị trí cảm biến tại bất kỳ bước thời gian nào một cách tự nhiên và do đó tạo điều kiện bù đắp biến dạng chuyển động trên cơ sở từng điểm. Nó cũng cho phép tích hợp các ràng buộc IMU mà không cần pre-integration bằng cách xem xét các đạo hàm của vị trí được nội suy.

Tuy nhiên, chúng tôi tránh biểu diễn liên tục theo thời gian

và sử dụng biểu diễn rời rạc theo thời gian thông thường trong công trình này vì hai lý do. Thứ nhất, các cảm biến range mới nổi không có cơ chế quét laser cung cấp point cloud không biến dạng với các điểm được thu thập đồng thời, làm cho continuous-time deskewing không cần thiết (ví dụ: Microsoft Azure Kinect, Intel Realsense, và solid state LiDARs). Thứ hai, trong bối cảnh visual SLAM, đã được báo cáo rằng biểu diễn liên tục theo thời gian không cải thiện độ chính xác ước lượng so với đối tác rời rạc, miễn là camera được đồng bộ hóa tốt với IMU [32]. Do đó, chúng tôi coi biểu diễn liên tục theo thời gian là không cần thiết khi biến dạng point cloud không đáng kể và có thể được loại bỏ đủ bằng dự đoán chuyển động dựa trên IMU.

Proposal: Chúng tôi đề xuất một thuật toán ước lượng odometry range-IMU liên kết chặt chẽ dựa trên keyframe-based point cloud matching và fixed-lag smoothing. Khác với các phương pháp dựa trên filtering thông thường chỉ tối ưu hóa trạng thái cảm biến mới nhất, phương pháp được đề xuất giữ mỗi frame hoạt động trong vài giây và cho phép xử lý sự suy thoái tạm thời của dữ liệu range bằng cách lan truyền các trạng thái cảm biến của các frame liên tiếp đến các frame quá khứ. Để làm cho framework tổng quát (tức là phù hợp với bất kỳ cảm biến range nào), chúng tôi sử dụng phương pháp đăng ký point cloud trực tiếp dựa trên so sánh distribution-to-distribution. Mặc dù các lựa chọn thiết kế này dẫn đến một lượng tính toán khổng lồ, chúng tôi chứng minh rằng có thể chạy thuật toán theo thời gian thực bằng cách tận dụng tối đa một GPU hiện đại.

B. Global Trajectory Optimization

Pose graph optimization: Tối ưu hóa pose graph xây dựng một factor graph với các ràng buộc vị trí tương đối (SE3) và tối ưu hóa các vị trí cảm biến bằng cách giảm thiểu các lỗi trong không gian pose. Cách tiếp cận này đã được sử dụng rộng rãi cho tối ưu hóa quỹ đạo toàn cục trong nhiều hệ thống SLAM nhờ tính đơn giản và hiệu quả của nó [9], [10], [17]. Tối ưu hóa pose graph mô hình hóa mỗi ràng buộc vị trí tương đối như một phân phối Gaussian trên manifold SE3. Tuy nhiên, trong thực tế, rất khó để ước lượng chính xác ma trận hiệp phương sai của một vị trí tương đối được cho bởi scan matching. Mặc dù một phương pháp closed-form thường được sử dụng để ước lượng ma trận hiệp phương sai từ ma trận Hessian của kết quả scan matching ICP [33], cách tiếp cận này có xu hướng lạc quan vì nó chỉ xem xét hàm mục tiêu tuyến tính hóa cuối cùng và không thể tính đến các thay đổi tương ứng điểm [7]. Mặc dù các phương pháp ước lượng hiệp phương sai dựa trên Monte-Carlo [34] chính xác hơn phương pháp dựa trên Hessian, chúng yêu cầu scan matching được thực hiện nhiều lần, dẫn đến chi phí tính toán lớn. Landry et al. đề xuất một phương pháp dựa trên học máy lấy các bộ mô tả point cloud và ước lượng ma trận hiệp phương sai dựa trên mô hình được huấn luyện offline. Mặc dù cách tiếp cận này nhằm cân bằng độ chính xác ước lượng và chi phí xử lý, độ chính xác ước lượng của nó có thể suy giảm trong môi trường chưa thấy. Hầu hết các framework SLAM dựa trên range hiện có chỉ sử dụng ma trận hiệp phương sai hằng số [12], một sơ đồ trọng số đơn giản [9], [10], hoặc ước lượng hiệp phương sai closed-form dựa trên Hessian [35]. Việc mô hình hóa ràng buộc không chính xác như vậy làm suy giảm độ chính xác ước lượng của các quỹ đạo dài với các vòng lặp lớn.

Đối với các point cloud chỉ có độ chồng lấp nhỏ, rất khó để xác định rõ ràng vị trí tương đối giữa các point cloud thông qua scan matching. Do đó, các framework hiện có, dựa vào tối ưu hóa pose graph, chỉ đóng vòng lặp khi có độ chồng lấp đủ giữa các point cloud, và loại bỏ thông tin về các cặp point cloud với độ chồng lấp nhỏ. Nếu chúng ta cưỡng bức đóng vòng lặp giữa các frame như vậy, các kết quả scan matching bị hỏng sẽ biến thành các ràng buộc tương đối không chính xác và dẫn đến ước lượng quỹ đạo suy giảm [36].

Bundle adjustment: Bundle adjustment (BA) là một cách tiếp cận để đồng thời ước lượng các tham số môi trường (ví dụ: vị trí landmark 3D) và trạng thái cảm biến qua các frame. Đây là một kỹ thuật thiết yếu cho visual SLAM và được sử dụng rộng rãi cho cả ước lượng odometry [17] và tối ưu hóa toàn cục [19]. Đối với SLAM dựa trên range, bài toán BA thường được xây dựng như tối ưu hóa chung của các tham số môi trường đường và mặt phẳng và vị trí cảm biến [37], [38]. Liu và Zhang chỉ ra rằng các tham số đường và mặt phẳng có thể được loại bỏ khỏi các biến tối ưu hóa vì chúng có thể được xác định từ tọa độ điểm và vị trí cảm biến, và xây dựng BA đường và mặt phẳng như giảm thiểu eigenvalue của các điểm tích lũy [39]. Huang et al. tiếp tục xây dựng lại bài toán BA mặt phẳng dưới dạng least-squares cho tối ưu hóa hiệu quả [40]. Mặc dù các phương pháp dựa trên BA hứa hẹn kết quả mapping nhất quán, chúng cần một ước đoán ban đầu tốt về vị trí cảm biến cho ước lượng tương ứng. Đặc biệt, công thức BA least squares [40] nhạy cảm với ước đoán ban đầu vì ước lượng pháp tuyến bề mặt của nó được tách biệt khỏi tối ưu hóa vị trí cảm biến. Các phương pháp dựa trên BA do đó gặp khó khăn trong việc đóng vòng lặp dưới độ trôi ước lượng lớn và cần được kết hợp với tối ưu hóa phân cấp với pose graph [41].

Global matching cost minimization: Lu và Milios xây dựng bài toán mapping dựa trên range 2D như giảm thiểu các lỗi đăng ký scan trên một factor graph [42]. Một số công trình mở rộng cách tiếp cận này lên ba chiều bằng cách xử lý rõ ràng các sự kiện đóng vòng lặp để giảm các lần thực hiện tối ưu hóa [43], [44]. Vì cách tiếp cận này không yêu cầu rõ ràng vị trí tương đối giữa các frame (nó chỉ đánh giá lỗi đăng ký giữa chúng), nó có thể tự nhiên lan truyền sự không chắc chắn khớp điểm đến sự không chắc chắn frame và ràng buộc chính xác vị trí tương đối giữa các frame chỉ với độ chồng lấp nhỏ. Tuy nhiên, cách tiếp cận này tốn kém về mặt tính toán vì các lỗi đăng ký cần được đánh giá lại trên toàn bộ bản đồ trong mỗi lần lặp tối ưu hóa và do đó giảm thiểu lỗi đăng ký toàn cục theo thời gian thực được coi là không khả thi.

Reijgwart et al. đề xuất một framework mapping thể tích dựa trên signed distance function submaps [45]. Framework của họ tạo ra các submap cục bộ dưới dạng Euclidean signed distance field (ESDF) và tối ưu hóa các vị trí submap sao cho các lỗi đăng ký giữa chúng được giảm thiểu. Vì khoảng cách giữa một điểm và bề mặt submap có thể được thu được trực tiếp với ESDF, tính toán lỗi đăng ký hiệu quả có thể được thực hiện mà không cần tìm kiếm láng giềng gần nhất tốn kém. Tuy nhiên, vẫn tồn tại vấn đề về mặt tính toán để tính toán lỗi đăng ký toàn cục và tối ưu hóa được thực hiện chỉ với một tập con ngẫu nhiên của các lỗi đăng ký với sự hỗ trợ của các ràng buộc vị trí tương đối SE3.

IMU constraints in global optimization: Xem xét rằng dữ

liệu IMU cung cấp thông tin về hướng trọng lực và ước chế độ trôi ước lượng odometry trong xoay yaw và pitch, người ta có thể thực hiện tối ưu hóa quỹ đạo toàn cục với tối ưu hóa pose graph bốn DoF để giữ kết quả tối ưu hóa toàn cục căn chỉnh trọng lực [17]. Tuy nhiên, kết quả ước lượng odometry có thể không được căn chỉnh hoàn hảo với hướng trọng lực do lỗi ước lượng bias. Trong những trường hợp như vậy, tối ưu hóa toàn cục bốn DoF sẽ dẫn đến kết quả mapping không nhất quán.

Chúng ta cũng có thể xem xét việc chèn trực tiếp các ràng buộc IMU vào factor graph tối ưu hóa toàn cục [46], [47]. Tuy nhiên, keyframes (hoặc submaps) cho tối ưu hóa toàn cục thường được tạo ra với khoảng thời gian lớn hơn. Nếu chúng ta đơn giản tạo một IMU factor giữa các keyframes, chúng ta không thể ràng buộc mạnh mẽ vị trí của chúng vì thời gian tích hợp dài hơn dẫn đến sự không chắc chắn lớn hơn của ràng buộc IMU [46]. Do đó, cách tiếp cận này yêu cầu việc tạo keyframes thường xuyên, dẫn đến tính toán dư thừa và hạn chế của thiết kế factor graph.

Usenko et al. đề xuất một kỹ thuật khôi phục factor phi tuyến để chuyển thông tin tích lũy trong quá trình visual inertial odometry sang tối ưu hóa toàn cục [48]. Trong kỹ thuật này, một tập hợp các factor phi tuyến được tạo ra để xấp xỉ một subgraph tuyến tính hóa (Markov bracket) của factor graph ước lượng odometry để biểu diễn phân phối gốc với topology graph thưa. Tuy nhiên, vì cách tiếp cận này khôi phục một subgraph tuyến tính hóa, các lỗi tuyến tính hóa là không thể tránh khỏi.

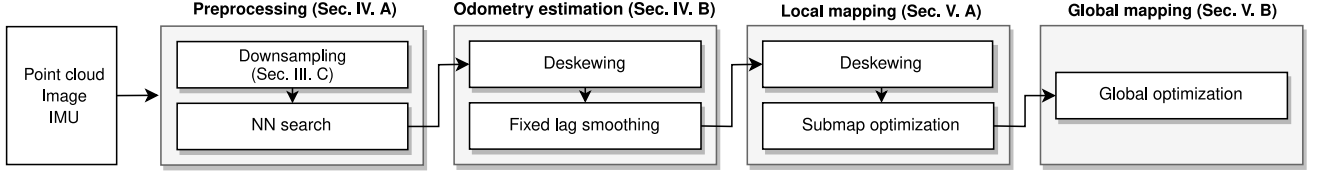
Proposal: So với cách tiếp cận tối ưu hóa pose graph thông thường, cách tiếp cận giảm thiểu lỗi đăng ký toàn cục được đề xuất cho phép vị trí tương đối giữa các submaps với độ chồng lấp rất nhỏ được ràng buộc chính xác, dẫn đến kết quả mapping nhất quán cao. Như trong thuật toán ước lượng odometry được đề xuất, chúng tôi tận dụng xử lý song song GPU để thực hiện giảm thiểu chi phí khớp toàn cục theo thời gian thực. Hơn nữa, chúng tôi giới thiệu khái niệm *endpoints* của submap để giữ thời gian tích hợp cho mỗi IMU factor nhỏ và ràng buộc mạnh mẽ các vị trí submap với thông tin quán tính, thường không được sử dụng trong các công trình hiện có.

III. PROPOSED SYSTEM

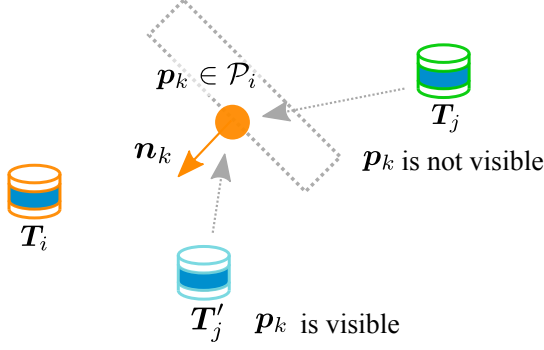
Chúng tôi đưa ra tổng quan về framework được đề xuất trong Sec. III-A. Sau đó chúng tôi mô tả ký hiệu và các khối xây dựng cho các module ước lượng trong Sec. III-B đến Sec. III-D. Cuối cùng, chúng tôi trình bày các thuật toán ước lượng odometry và tối ưu hóa toàn cục được đề xuất trong Sec. IV và Sec. V.

A. System Overview

Hình 1 cho thấy tổng quan của framework được đề xuất. Hệ thống mapping bao gồm một module tiền xử lý và ba module ước lượng, cụ thể là ước lượng odometry, mapping cục bộ, và mapping toàn cục. Tất cả các module ước lượng đều dựa trên liên kết chặt chẽ của các ràng buộc range và IMU. Tùy chọn, thông tin hình ảnh đa camera được tích hợp trong module ước lượng odometry để cải thiện tính ổn định và độ chính xác. Chúng tôi thiết kế framework để đơn khối cho hiệu quả; mỗi module chạy trong một thread riêng biệt và toàn bộ hệ thống được thực thi như một process đơn lẻ.



Hình 1. Framework được đề xuất bao gồm module tiền xử lý và ba module ước lượng (odometry estimation, local mapping, và global mapping) đều dựa trên tight coupling của range và IMU constraints. Các point cloud đầu vào được đưa vào module tiền xử lý để downsampling và tìm kiếm nearest neighbor. Sau đó, module odometry estimation thực hiện deskewing point cloud để loại bỏ motion distortion, và chuyển động cảm biến được ước lượng thông qua fixed-lag smoothing. Module local mapping thực hiện deskewing lại với kết quả odometry estimation và ghép nối một số frames thành một submap sau local map optimization. Cuối cùng, module global mapping lấy submaps làm đầu vào và tối ưu hóa poses của chúng để cải thiện global consistency. Mỗi module chạy trong một thread riêng biệt và toàn bộ hệ thống được thực thi như một process đơn lẻ.



Hình 2. Xác thực tương ứng dựa trên định hướng bề mặt. $p_k \in \mathcal{P}_i$ không thể nhìn thấy từ T_j , ở phía bên kia của bề mặt, nhưng có thể nhìn thấy từ T'_j , ở cùng phía của bề mặt.

B. Notation

Chúng tôi định nghĩa trạng thái cảm biến x_t tại thời điểm t sẽ được ước lượng trong các module ước lượng như

$$x_t = [T_t, v_t, b_t], \quad (1)$$

trong đó $T_t = [R_t | t_t] \in \text{SE}(3)$ là vị trí cảm biến, $v_t \in \mathbb{R}^3$ là vận tốc, và $b_t = [b_t^a, b_t^\omega] \in \mathbb{R}^6$ là bias gia tốc tuyến tính và vận tốc góc của IMU. Chúng tôi ước lượng chuỗi thời gian của các trạng thái cảm biến từ point clouds $\mathcal{P}_t$, các phép đo IMU (gia tốc tuyến tính a_t và vận tốc góc ω_t), và hình ảnh camera tùy chọn I_t . Lưu ý rằng chúng tôi giả định rằng các phép biến đổi giữa cảm biến range, IMU, và camera là đã biết. Chúng tôi biến đổi point clouds vào frame IMU để xử lý chúng như thể chúng đang ở trong một hệ tọa độ cảm biến thống nhất cho hiệu quả và đơn giản.

C. Matching Cost Factor

Matching cost factor ràng buộc hai vị trí cảm biến (T_i và T_j) sao cho lỗi đăng ký giữa các point cloud ($\mathcal{P}_i$ và $\mathcal{P}_j$) được giảm thiểu. Như matching cost, chúng tôi chọn voxelized GICP (VGICP) [49], là một biến thể của GICP [16] với data association dựa trên voxel. Để cải thiện độ chính xác ước lượng trong môi trường trong nhà, chúng tôi mở rộng implementation VGICP trước đây [49] với xác thực tương ứng dựa trên định hướng bề mặt và một voxelmap đã độ phân giải.

Preprocessing: VGICP mô hình hóa mỗi điểm đầu vào $p_k \in \mathcal{P}_i$ như một phân phối Gaussian $p_k = (\mu_k, C_k)$, biểu diễn hình

dạng cục bộ của bề mặt xung quanh p_k . Ma trận hiệp phương sai C_k được tính từ các điểm lân cận của p_k được cho bởi tìm kiếm k-nearest-neighbor. Sau đó chúng tôi tạo ra một voxelmap thưa với spatial voxel hashing [50] và lấy trung bình của các điểm và hiệp phương sai của chúng trong mỗi voxel. Để mở rộng basin of convergence, chúng tôi tạo ra các voxelmap với L mức độ phân giải (ví dụ: ba mức) được chia tỷ lệ theo lũy thừa của hai; kích thước voxel của voxelmap thứ l được cho bởi $r^l = r_0 2^{(l-1)}$, trong đó r_0 là kích thước voxel cơ sở. Một điểm đầu vào được liên kết với các voxel tương ứng trong mỗi mức độ phân giải để tính khoảng cách point-to-voxel.

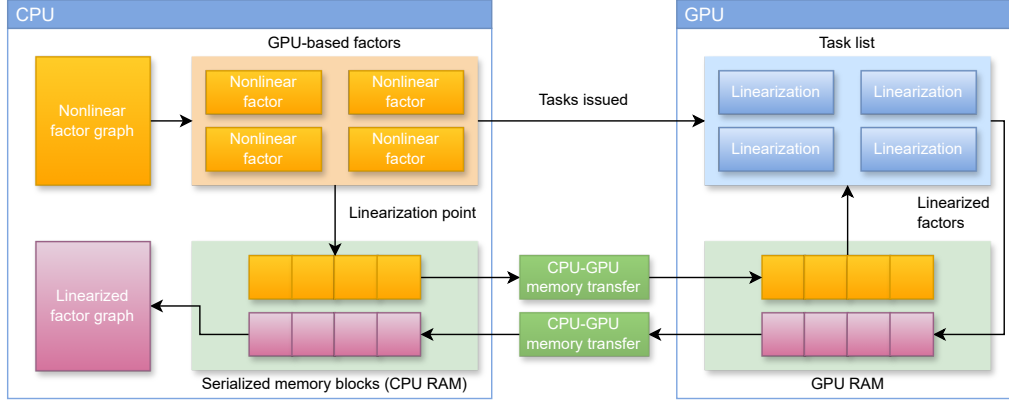
Lưu ý rằng so sánh một điểm và phân phối trung bình của một voxel tương đương với việc so sánh điểm và tất cả các điểm trong voxel trong một so sánh distribution-to-distribution, như được thể hiện trong [49]. Vì chúng ta có thể thu được một phân phối hợp lệ ngay cả trên một voxel xa với chỉ một vài điểm, cách tiếp cận này dẫn đến độ chính xác đăng ký tốt hơn so với normal distributions transform (NDT) [51], cần nhiều điểm cho mỗi voxel để thu được phân phối voxel phù hợp.

Correspondence search: Trong môi trường trong nhà, cả hai mặt của một bức tường mỏng thường được quan sát khi cảm biến di chuyển. Trong những trường hợp như vậy, tìm kiếm tương ứng dựa trên khoảng cách gần có thể liên kết sai các điểm ở một mặt của tường với các điểm ở mặt bên kia. Để tránh điều này, chúng tôi kiểm tra xem mỗi điểm đầu vào $p_k \in \mathcal{P}_i$ có thể nhìn thấy từ góc nhìn của point cloud khác $\mathcal{P}_j$ dựa trên pháp tuyến bề mặt n_k hay không, như thể hiện trong Hình 2. Nếu $(p_k - T_i^{-1}t_j) \cdot n_k > 0$, T_j ở phía bên kia của bề mặt và điểm được coi là không thể nhìn thấy từ T_j và bị loại bỏ. Ngược lại, chúng tôi coi rằng p_k có thể nhìn thấy từ T_j và tính toán tọa độ voxel tương ứng $q_k = \text{floor}(p_k/r) \in \mathbb{Z}^3$, trong đó r là độ phân giải voxel. Các voxel tương ứng ở mỗi mức độ phân giải voxel được tra cứu từ các voxelmap của $\mathcal{P}_j$. Xác thực tương ứng đơn giản này hoạt động hiệu quả với ước lượng odometry dựa trên so sánh frame-by-frame được đề xuất trong Sec. IV.

Linearization: Như hàm mục tiêu cần giảm thiểu, chúng tôi tính tổng khoảng cách distribution-to-distribution giữa các điểm $p_k = (\mu_k, C_k)$ và các voxel tương ứng của chúng $\tilde{p}_k^l = (\tilde{\mu}_k^l, \tilde{C}_k^l)$ như sau:

$$e^{PC}(T_i, T_j) = \sum_{l \in [1 \dots L]} \sum_{p_k \in \mathcal{P}_i} e^{D2D}(p_k, \tilde{p}_k^l, T_i^{-1}T_j), \quad (2)$$

$$e^{D2D}(p_k, \tilde{p}_k^l, T_{ij}) = d_k^T \left(\tilde{C}_k^l + T_{ij} C_k T_{ij}^T \right)^{-1} d_k, \quad (3)$$



Hình 3. Tuyến tính hóa factor hiệu quả trên GPU. Dữ liệu đầu vào và đầu ra cho tuyến tính hóa được serialize trong một memory block và upload lên và download từ GPU với một lần chuyển memory duy nhất. Các task tuyến tính hóa được đưa vào GPU task list và xử lý song song. Cơ chế này chỉ gọi hai lần chuyển dữ liệu CPU-GPU cho mỗi task tuyến tính hóa factor graph bất kể số lượng factors trong factor graph.

trong đó $T_{ij} = T_i^{-1}T_j$ là vị trí tương đối giữa T_i và T_j và $d_k = \tilde{\mu}_k^l - T_{ij}\mu_k$ là phần dư giữa μ_k và $\tilde{\mu}_k^l$. Như trong GICP gốc, chúng tôi giả định rằng các độ dịch chuyển của T_i và T_j là nhỏ trong mỗi lần lặp tối ưu hóa và sử dụng $\Omega_k = (\tilde{C}_k^l + T_{ij}C_kT_{ij}^T)^{-1}$ cố định tại điểm tuyến tính hóa để biến Eq. 3 thành dạng weighted least squares.

Vì e^{PC} ở dạng weighted least squares, chúng tôi có thể suy ra một linear quadratic factor, bao gồm các ma trận thông tin H_{ii} , H_{ij} , và H_{jj} và các vector thông tin b_i và b_j , từ các đạo hàm bậc nhất:

$$A_k = \frac{\partial d_k}{\partial T_i} = [R_{ij}(\mu_k)_\times, -R_{ij}], \quad (4)$$

$$B_k = \frac{\partial d_k}{\partial T_j} = [-(T_{ij}p_k)_\times, \mathbb{I}_{3 \times 3}], \quad (5)$$

$$H_{ii} = \sum_k^N A_k \Omega_k A_k^T, \quad H_{ij} = \sum_k^N A_k \Omega_k B_k^T, \quad (6)$$

$$H_{jj} = \sum_k^N B_k \Omega_k B_k^T, \quad (7)$$

$$b_i = \sum_k^N A_k \Omega_k d_k, \quad b_j = \sum_k^N B_k \Omega_k d_k, \quad (8)$$

trong đó $()_\times$ biến đổi một vector thành ma trận phản đối xứng (tức là hat map).

Implementation: Tìm kiếm tương ứng và tuyến tính hóa có thể được tính toán hiệu quả trên GPU vì chúng dựa trên tra cứu bộ nhớ đơn giản và các phép toán ma trận tuyến tính và không liên quan đến các nhánh điều kiện gây ra sự phân kỳ thread. Tuy nhiên, đơn giản tuyến tính hóa các factor từng cái một (trong một vòng lặp *for*) sẽ yêu cầu đồng bộ hóa CPU-GPU cho mỗi factor, điều này sẽ dẫn đến overhead lớn và sẽ ảnh hưởng đáng kể đến tốc độ xử lý.

Để tránh overhead đồng bộ hóa và tận dụng tối đa sức mạnh của GPU, chúng tôi đã phát triển một cơ chế tuyến tính hóa hiệu quả, được thể hiện trong Hình 3. Đầu tiên chúng tôi liệt kê tất cả các factor phi tuyến dựa trên GPU (matching cost factors trong

trường hợp của chúng tôi) và serialize tất cả dữ liệu cần thiết cho tuyến tính hóa (ví dụ: các điểm tuyến tính hóa) trong một khối bộ nhớ đơn lẻ. Khối bộ nhớ đầu vào đã serialize được tải lên GPU với một lần gọi chuyển bộ nhớ đơn lẻ. Sau đó, các tác vụ tuyến tính hóa được phát hành đến danh sách tác vụ GPU và được xử lý thông qua một số processing streams song song. Kết quả tuyến tính hóa được serialize vào một khối bộ nhớ trên GPU và gửi lại CPU sau khi đồng bộ hóa CPU-GPU. Implementation này giảm thiểu overhead GPU và tối đa hóa tốc độ xử lý vì nó chỉ gọi hai lần chuyển dữ liệu CPU-GPU cho mỗi tác vụ tuyến tính hóa factor graph bất kể số lượng factor cần tuyến tính hóa.

D. IMU Preintegration Factor

Chúng tôi sử dụng kỹ thuật IMU preintegration [28] để tích hợp hiệu quả các ràng buộc IMU vào factor graph. Cho một phép đo IMU (a_t và ω_t), trạng thái cảm biến phát triển theo thời gian như sau:

$$R_{t+\Delta t} = R_t \exp((\omega_t - b_t^\omega - \eta_t^\omega) \Delta t), \quad (9)$$

$$v_{t+\Delta t} = v_t + g\Delta t + R_t(a_t - b_t^a - \eta_t^a) \Delta t, \quad (10)$$

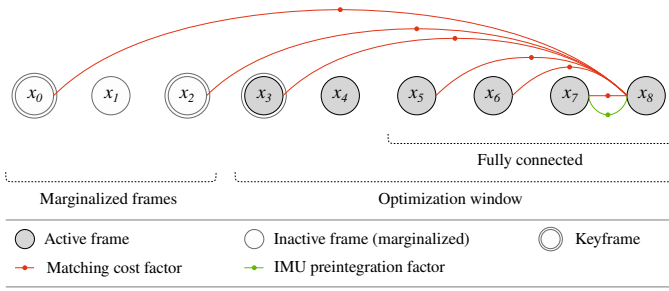
$$t_{t+\Delta t} = t_t + v_t \Delta t + \frac{1}{2}g\Delta t^2 + \frac{1}{2}R_t(a_t - b_t^a - \eta_t^a) \Delta t^2, \quad (11)$$

trong đó g là vector trọng lực và η_t^a và η_t^ω đại diện cho nhiễu trắng trong phép đo IMU.

IMU preintegration factor tích hợp sự phát triển hệ thống giữa hai bước thời gian i và j để thu được chuyển động cơ thể tương đối ΔR_{ij} , Δt_{ij} , và Δv_{ij} (xem [28] để biết chi tiết suy dẫn) để ràng buộc các trạng thái cảm biến:

$$e^{IMU}(x_i, x_j) = \|\log(\Delta R_{ij}^T R_i^T R_j)\|^2 + \|\Delta t_{ij} - R_i^T \left(t_j - t_i - v \Delta t_{ij} - \frac{1}{2}g\Delta t_{ij}^2 \right)\|^2 + \|\Delta v_{ij} - R_i^T (v_j - v_i - g\Delta t_{ij})\|^2. \quad (12)$$

IMU preintegration factor cho phép chúng tôi giữ factor graph được ràng buộc tốt trong môi trường mà các đặc trưng



Hình 4. Factor graph ước lượng odometry. Chỉ các factors liên quan đến frame mới nhất (x_8) được visualize. Frame mới nhất được kết nối với keyframes thông qua matching cost factors để giảm độ trôi ước lượng. Nó cũng được kết nối với một vài frame cuối để làm cho ước lượng mạnh mẽ đối với chuyển động cảm biến nhanh. IMU factors được tạo giữa các frame liên tiếp để giữ factor graph được ràng buộc tốt.

hình học không đủ và matching cost factors có thể bị suy thoái. Hơn nữa, nó cung cấp thông tin về hướng trọng lực và giảm độ trôi ước lượng trong bốn DoFs [17].

IV. ODOMETRY ESTIMATION

Trong phần này, chúng tôi trình bày thuật toán ước lượng odometry dựa trên tích hợp dữ liệu range và IMU. Sự khác biệt chính so với các phương pháp hiện có là thuật toán được đề xuất sử dụng kết hợp fixed-lag smoothing và keyframe-based point cloud matching thay vì frame-to-model matching và ước lượng dựa trên filtering. Fixed-lag smoothing liên tục cập nhật các trạng thái cảm biến khi chúng ở trong cửa sổ tối ưu hóa. Điều này có thể được coi là frame-to-model matching với một target model hoạt động. Cách tiếp cận này làm cho ước lượng mạnh mẽ đối với sự suy thoái tạm thời của dữ liệu range vì nó đặt các ước lượng trạng thái cảm biến trên hold trong vài giây và lan truyền ước lượng trạng thái cảm biến mới nhất đến các trạng thái quá khứ để sửa độ trôi ước lượng một khi các ràng buộc hình học đủ trên dữ liệu range trở nên có sẵn. Hơn nữa, không giống như các phương pháp dựa trên filtering, các biến và ràng buộc không cần được chèn theo thứ tự thời gian nghiêm ngặt; chúng ta có thể chèn các ràng buộc cho các trạng thái cảm biến quá khứ miễn là các trạng thái vẫn còn trong cửa sổ tối ưu hóa. Điều này cho phép chúng ta chạy bất đồng bộ xử lý dữ liệu bổ sung (ví dụ: visual feature tracking) và chèn các ràng buộc vào factor graph với một độ trễ nhỏ.

A. Preprocessing

Đầu tiên chúng tôi downsample input point cloud và tìm k điểm lân cận cho mỗi điểm cần thiết cho ước lượng hiệp phương sai điểm tiếp theo được thực hiện sau khi bù đắp biến dạng chuyển động. Chúng tôi giả định rằng mối quan hệ lân cận của các điểm không thay đổi đáng kể trong quá trình bù đắp biến dạng chuyển động và sử dụng các nearest neighbors được tính trước cho ước lượng hiệp phương sai để giảm thời gian xử lý. Lưu ý rằng tìm kiếm nearest neighbor chính xác tốn kém chỉ được thực hiện trong bước tiền xử lý; tất cả các bước ước lượng tiếp theo sử dụng tìm kiếm nearest neighbor xấp xỉ dựa trên voxel.

B. Tightly Coupled Range-IMU Odometry Estimation

Đầu tiên chúng tôi sửa biến dạng của point cloud gây ra bởi chuyển động cảm biến bằng cách biến đổi các điểm vào frame IMU với dự đoán chuyển động dựa trên động lực học IMU. Sau đó chúng tôi tính hiệp phương sai của mỗi điểm bằng cách sử dụng các điểm lân cận được tính trước.

Cho các point clouds và phép đo IMU, chúng tôi xây dựng factor graph được thể hiện trong Hình 4. Để giới hạn chi phí tính toán và đảm bảo rằng thuật toán ước lượng odometry có khả năng thời gian thực, chúng tôi sử dụng cách tiếp cận fixed-lag smoothing và loại bỏ các frame cũ di chuyển ra khỏi cửa sổ tối ưu hóa. Chúng tôi sử dụng iSAM2 [52] và variable elimination dựa trên Bayes-tree hiệu quả của nó cho fixed-lag smoothing được implement trong GTSAM [53].

Lấy cảm hứng từ direct sparse odometry [54], chúng tôi giới thiệu một cơ chế keyframe cho ước lượng quỹ đạo hiệu quả và ít trôi. Keyframes là một tập hợp các frame được chọn sao cho chúng được phân bố không gian tốt trong khi có độ chồng lấp đủ với frame mới nhất. Chúng tôi tạo ra một matching cost factor giữa frame mới nhất và mỗi keyframe để giảm hiệu quả độ trôi ước lượng. Nếu một keyframe đã được loại bỏ bởi fixed-lag smoother, chúng tôi coi vị trí keyframe là cố định và tạo ra một matching cost factor ràng buộc vị trí cảm biến mới nhất đối với keyframe cố định.

Để quản lý keyframes, chúng tôi định nghĩa overlap rate giữa hai point cloud $\mathcal{P}_i$ và $\mathcal{P}_j$ như phần của các điểm trong $\mathcal{P}_i$ rơi vào một voxel của $\mathcal{P}_j$ [36]. Bất cứ khi nào một point cloud frame mới đến, chúng tôi đánh giá overlap rate giữa frame đó và union của tất cả keyframes, và nếu độ chồng lấp nhỏ hơn một ngưỡng (ví dụ: 90%), chúng tôi chèn frame đó vào danh sách keyframe. Tương tự như chiến lược loại bỏ keyframe trong [54], chúng tôi loại bỏ các keyframe dư thừa bằng cách sử dụng chiến lược sau:

- 1) Chúng tôi loại bỏ các keyframes chồng lấp keyframe mới nhất ít hơn một ngưỡng nhất định (ví dụ: 5%).
- 2) Nếu nhiều hơn N^{odom} (ví dụ: 20) frame tồn tại trong danh sách keyframe, chúng tôi loại bỏ keyframe giảm thiểu score sau:

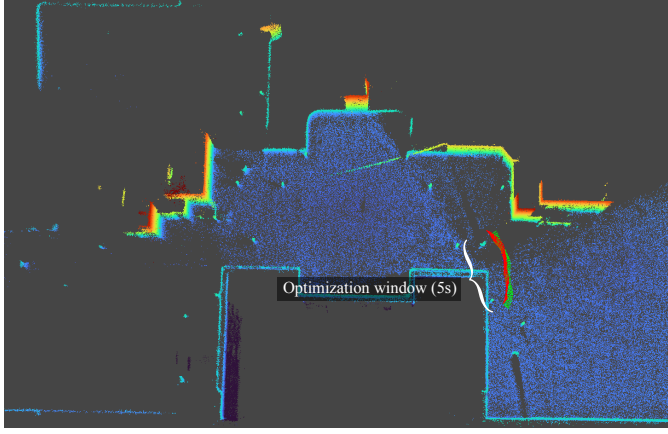
$$s(i) = o(i, N^{odom}) \sum_{j \in [1, N^{odom} - 1] \setminus \{i\}} (1 - o(i, j)), \quad (13)$$

trong đó $o(i, j)$ là overlap rate giữa keyframe thứ i và thứ j . Hàm score được thiết kế theo kinh nghiệm để giữ keyframes phân bố không gian tốt trong khi để lại nhiều keyframes gần với keyframe mới nhất.

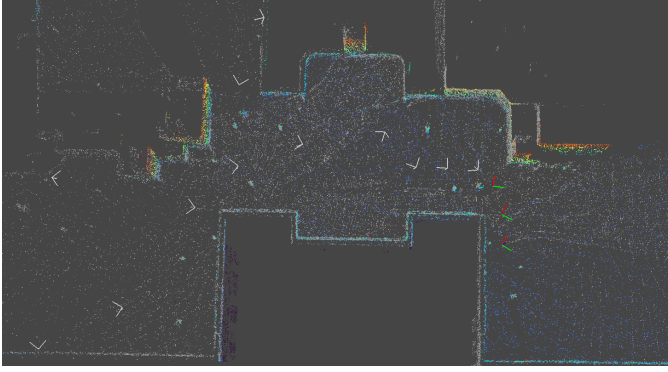
Ngoài các keyframes, chúng tôi tạo ra các matching cost factors giữa frame mới nhất và N^{pre} frame cuối (ví dụ: ba frame cuối) để làm cho ước lượng odometry mạnh mẽ đối với chuyển động cảm biến nhanh. Chúng tôi cũng tạo ra một IMU preintegration factor giữa các frame liên tiếp để mạnh mẽ trong môi trường không có đặc trưng.

Tóm lại, hàm mục tiêu của thuật toán ước lượng odometry được cho bởi:

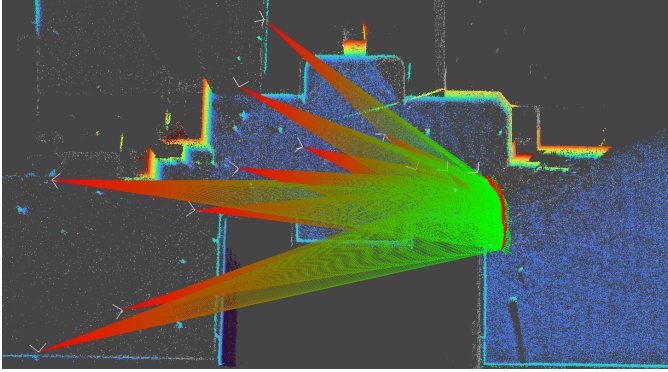
$$g^{LIO}(\mathcal{X}^a) = \sum_{\mathbf{x}_i \in \mathcal{X}^a} \sum_{\mathbf{x}_j \in \mathcal{X}^p \cup \mathcal{X}^k} e^{PC}(\mathcal{P}_i, \mathcal{P}_j, \mathbf{T}_i, \mathbf{T}_j) + \sum_{\mathbf{x}_i \in \mathcal{X}^a} e^{IMU}(\mathbf{x}_{i-1}, \mathbf{x}_i) + e^{MG}(\mathcal{X}^a), \quad (14)$$



(a) Active frames



(b) Keyframes

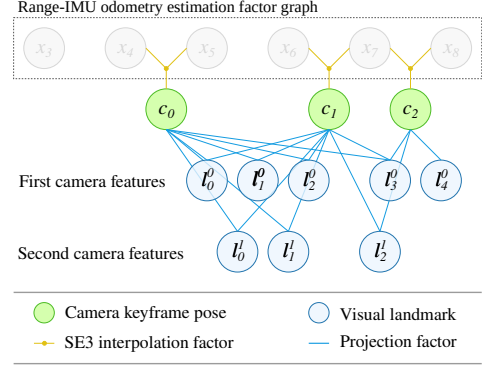


(c) Matching cost factors

Hình 5. Ví dụ về keyframes và matching cost factors được tạo ra. Coordinate frames và points được thể hiện bằng màu đại diện cho active frames trong cửa sổ tối ưu hóa (5 s) và những cái được thể hiện bằng màu xám là cố định. Các matching cost factors được tạo giữa active frames và keyframes được chọn để phân bố không gian tốt để giảm hiệu quả độ trôi ước lượng.

trong đó $\mathcal{X}^a$ là tập hợp các active frames trong cửa sổ tối ưu hóa, $\mathcal{X}_i^p = [\mathbf{x}_{i-N^{pre}}, \dots, \mathbf{x}_{i-1}]$ là các frame trước đó của $\mathbf{x}_i$, $\mathcal{X}^k$ là các keyframes, và e^{MG} là error term để bù đắp cho các biến và factors đã loại bỏ.

Hình 5 thể hiện các ví dụ về active frames, keyframes, và matching cost factors. Coordinate frames và points được thể hiện bằng màu đại diện cho active frames trong cửa sổ tối ưu hóa của fixed-lag smoother (5 s) và point clouds. Những cái được thể hiện bằng màu xám là các keyframes và point clouds cố định (đã loại bỏ). Hình 5 (a) thể hiện active frames chịu tối ưu hóa và Hình 5 (b) thể hiện keyframes. Chúng ta có thể thấy rằng các keyframes được chọn sao cho chúng được phân



Hình 6. Extension đa camera cho odometry estimation factor graph. Camera keyframe poses được giới thiệu như nội suy của IMU poses và được ràng buộc bởi visual feature projection factors.

bổ không gian tốt để giảm hiệu quả độ trôi ước lượng. Một vài keyframes đầu tiên vẫn còn trong cửa sổ tối ưu hóa và được coi là active. Hình 5 (c) thể hiện matching cost factors được tạo dày đặc giữa các active frames và keyframes. Mặc dù khó xác nhận bằng mắt, cũng có các matching cost factors giữa các active frames, như trong Hình 4, làm cho ước lượng mạnh mẽ đối với chuyển động cảm biến nhanh. Đáng chú ý rằng chúng tôi đánh giá lại mỗi matching cost factor (mỗi đường trong Hình 5 (c)) trong mỗi lần lặp tối ưu hóa và giảm thiểu tổng lỗi đăng ký trên factor graph được kết nối dày đặc trong thời gian thực.

C. Tightly Coupled Multi-Camera Visual Constraints

Chúng tôi thiết kế framework được đề xuất để modular sao cho nó có thể chấp nhận các ràng buộc bổ sung để cải thiện thêm tính ổn định và độ chính xác ước lượng. Ở đây, chúng tôi trình bày một extension đa camera cho tightly coupled visual-range-IMU odometry estimation.

Hình 6 thể hiện odometry estimation factor graph được mở rộng với multi-camera visual constraints. Chúng tôi giả định rằng các cameras được hardware-synchronized và chụp hình ảnh cùng lúc. Cho mỗi camera, chúng tôi phát hiện và theo dõi các điểm đặc trưng 2D bằng cách sử dụng curvature extreme tracking [55]. Bất cứ khi nào range sensor đo một lượng displacement nhất định (ví dụ: 0.25 m hoặc 15°) hoặc thời gian kể từ lần tạo visual keyframe cuối vượt quá ngưỡng (ví dụ: 3 s), chúng tôi tạo ra một visual keyframe $\mathbf{C}_t \in \text{SE}(3)$ đại diện cho IMU pose tại thời điểm khi một bộ hình ảnh được chụp. Để ràng buộc $\mathbf{C}_t$, chúng tôi nội suy hai IMU poses gần nhất $\mathbf{T}_L = [\mathbf{R}_L | \mathbf{t}_L]$ và $\mathbf{T}_R = [\mathbf{R}_R | \mathbf{t}_R]$ bao phủ thời điểm của $\mathbf{C}_t$ bằng cách sử dụng spherical linear interpolation (SLERP):

$$\mathbf{t}_t^c = (1 - \alpha)\mathbf{t}_L + \alpha\mathbf{t}_R, \quad (15)$$

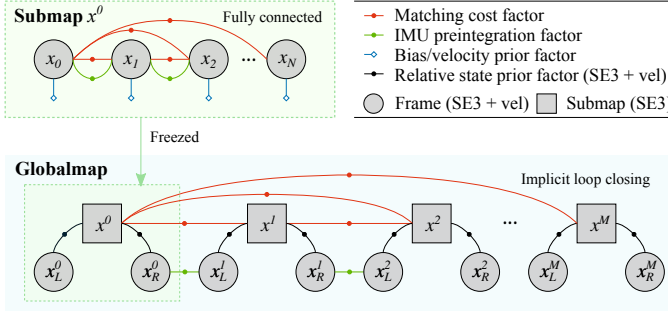
$$\mathbf{R}_t^c = \text{SLERP}(\mathbf{R}_L, \mathbf{R}_R, \alpha), \quad (16)$$

$$\alpha = \frac{t^C - t^L}{t^R - t^L}, \quad (17)$$

$$e^{VK}(\mathbf{C}_t) = \|\log(\mathbf{C}_t^{-1}[\mathbf{R}_t^c | \mathbf{t}_t^c])\|^2, \quad (18)$$

trong đó $t^L < t^C < t^R$ là các điểm thời gian của $\mathbf{T}_L$, $\mathbf{C}_t$, và $\mathbf{T}_R$, tương ứng.

Mỗi khi một visual keyframe được tạo ra, chúng tôi trích xuất các đặc trưng hình ảnh xuất hiện trong cả visual keyframe trước



Hình 7. Factor graph cho global optimization. Module local mapping tinh chỉnh kết quả ước lượng và hợp nhất một số local frames thành một submap sử dụng chiến lược đăng ký all-to-all. Module global mapping tối ưu hóa submap poses sao cho lỗi đăng ký toàn cục được giảm thiểu trên toàn bộ bản đồ. Cả hai module tận dụng IMU factors để ổn định hóa ước lượng trong môi trường không có đặc trưng nghiêm trọng và giảm độ trôi ước lượng.

đó và hiện tại. Sau đó chúng tôi lọc ra các đặc trưng hình ảnh outlier thông qua xác thực đặc trưng hai giai đoạn. Nó đầu tiên loại bỏ outliers bằng cách sử dụng ước lượng essential matrix dựa trên RANSAC và sau đó triangulate các vị trí đặc trưng 3D dựa trên sensor poses được dự đoán bởi LiDAR-IMU fusion. Các đặc trưng hình ảnh với reprojection errors lớn hơn ngưỡng sau đó bị loại bỏ. Cho mỗi đặc trưng hình ảnh vượt qua xác thực, chúng tôi tạo ra một projection factor để ràng buộc vị trí landmark tương ứng và visual keyframe pose.

Cho một điểm đặc trưng hình ảnh 2D $f_t^j \in \mathbb{R}^2$ được theo dõi bởi camera thứ i tại thời điểm t với feature ID j , chúng tôi tạo ra một projection factor với Huber's robust kernel được định nghĩa như:

$$e^{VF}(C_t, l_i^j) = \|f_t^j - \rho\left(\pi\left(C_t T_{C_i}^I l_i^j\right)\right)\|^2, \quad (19)$$

trong đó $l_i^j \in \mathbb{R}^3$ là vị trí visual landmark thứ j liên kết với camera thứ i , $T_{C_i}^I$ là phép biến đổi hằng số giữa frame IMU và frame quang học của camera thứ i , π là hàm projection, và ρ là robust kernel.

Hàm mục tiêu cho odometry estimation được mở rộng với visual constraints được định nghĩa như:

$$g^{LVIO}(\mathcal{X}^a, \mathcal{C}^a, \mathcal{L}^a) = g^{LIO}(\mathcal{X}^a) + \sum_{C_t \in \mathcal{C}^a} e^{VK}(C_t) + \sum_{i=1}^{N^{CAM}} \sum_{C_t \in \mathcal{C}^a} \sum_{l_i^j \in \mathcal{L}_t^i} e^{VF}(C_t, l_i^j), \quad (20)$$

trong đó $\mathcal{C}^a$ và $\mathcal{L}^a$ tương ứng là các tập hợp visual keyframe poses và visual landmarks trong cửa sổ tối ưu hóa, và $\mathcal{L}_t^i$ là tập hợp các visual landmarks có observation trên hình ảnh camera thứ i tại thời điểm t . Như thể hiện trong Hình 6, tất cả các biến được tối ưu hóa chung trên một factor graph duy nhất bằng cách xem xét multi-sensor constraints (tức là tightly coupled visual-range-IMU odometry estimation).

V. GLOBAL TRAJECTORY OPTIMIZATION

A. Local Mapping

Một khi một frame được loại bỏ từ odometry estimation graph, nó được đưa vào local mapping module như một ước

lượng ban đầu của trạng thái cảm biến. Local mapping module kết hợp một số local frames thành một submap để giảm số lượng biến được tối ưu hóa trong global mapping module.

Đầu tiên chúng tôi thực hiện deskewing và ước lượng hiệp phương sai lại với trạng thái đã loại bỏ, được mong đợi cải thiện từ dự đoán ban đầu được thực hiện ở đầu odometry estimation. Sau đó chúng tôi đánh giá overlap rate giữa frame đó và frame mới nhất trong submap, và nếu overlap rate nhỏ hơn một ngưỡng (ví dụ: 90%), chúng tôi chèn frame đó vào submap factor graph.

Như thể hiện trong Hình 7, chúng tôi tạo ra một matching cost factor cho mỗi tổ hợp frames trong submap (tức là all-to-all registration). Chúng tôi cũng thêm một IMU preintegration factor giữa các frame liên tiếp và thêm một prior factor cho vận tốc và bias của mỗi frame, dựa trên trạng thái đã loại bỏ, để ổn định hóa tối ưu hóa submap.

Hàm mục tiêu cho local mapping do đó được định nghĩa như:

$$g^{LM}(\mathcal{X}^s) = \sum_{(x_i, x_j) \in {}_2\mathcal{C}_{\mathcal{X}^s}} e^{PC}(T_i, T_j) + \sum_{x_i \in \mathcal{X}^s} e^{IMU}(x_{i-1}, x_i) + \sum_{x_i \in \mathcal{X}^s} e^{BV}(x_i), \quad (21)$$

trong đó $\mathcal{X}^s$ là tập hợp các frame trong submap, ${}_2\mathcal{C}_{\mathcal{X}^s}$ là tất cả các tổ hợp của các frame, $e^{BV}(x_i) = \|b_i - b'_i\|^2 + \|v_i - v'_i\|^2$ là một prior factor để giữ các ước lượng vận tốc và IMU bias gần với những giá trị của trạng thái đã loại bỏ (b'_i và v'_i).

Một khi số lượng frame trong submap trở nên bằng N^{sub} (ví dụ: 15), hoặc độ chồng lấp giữa frame đầu tiên và cuối cùng trở nên nhỏ hơn một ngưỡng (ví dụ: 5%), chúng tôi thực hiện factor graph optimization sử dụng Levenberg-Marquardt optimizer [56] và hợp nhất các frame thành một submap dựa trên kết quả tối ưu hóa. Ngược lại với odometry estimation, chúng tôi thực hiện batch optimization cho local mapping. Điều này cho phép chúng tôi sửa độ trôi ước lượng gây ra bởi một khoảng thời gian dài hơn của sự suy thoái dữ liệu range mà khó xử lý thông qua online optimization với cửa sổ tối ưu hóa bị giới hạn được sử dụng trong odometry estimation.

B. Global Mapping

Global mapping module tối ưu hóa các submap poses sao cho các lỗi đăng ký giữa chúng được giảm thiểu trên toàn bộ bản đồ. Nó tạo ra một matching cost factor giữa mỗi cặp submap với overlap rate vượt quá một ngưỡng nhỏ (ví dụ: 5%). Điều này dẫn đến một factor graph cực kỳ dày đặc. Bởi vì mỗi submap được căn chỉnh không chỉ với các submaps liền kề trên graph mà còn với mỗi submap được thăm lại, các vòng lặp quỹ đạo được đóng một cách ngầm định.

Submaps được tạo ra với khoảng thời gian lớn hơn. Nếu chúng tôi đơn giản tạo ra một IMU factor giữa các trạng thái submap, độ không chắc chắn của nó trở nên rất lớn và nó không thể ràng buộc mạnh mẽ relative pose giữa các submaps [18], [46]. Hơn nữa, chúng tôi cũng mất thông tin về vận tốc và IMU bias được ước lượng bởi odometry estimation module. Để giải quyết các vấn đề này, chúng tôi giới thiệu hai trạng thái gọi là endpoints (x_L^i và x_R^i) cho mỗi submap x^i ; chúng giữ các trạng thái của frame đầu tiên và cuối cùng trong submap đối với submap pose.

Cho một ước lượng của sensor states $[\mathbf{x}_1, \dots, \mathbf{x}_{N_{sub}}]$ trong một submap $\mathbf{x}^i$, chúng tôi định nghĩa submap origin $\mathbf{T}^i = [\mathbf{R}^i | \mathbf{t}^i]$ như pose của sensor tại trung tâm $\mathbf{T}_{N_{sub}/2}$. Sau đó, sensor state $\mathbf{x}_t$ tương đối với submap origin được cho như:

$$\mathbf{T}'_t = (\mathbf{T}^i)^{-1} \mathbf{T}_t, \quad (22)$$

$$\mathbf{v}'_t = (\mathbf{R}^i)^{-1} \mathbf{v}_t, \quad (23)$$

$$\mathbf{b}'_t = \mathbf{b}_t. \quad (24)$$

Chúng tôi coi $\mathbf{T}'_t$, $\mathbf{v}'_t$, và $\mathbf{b}'_t$ là cố định và tạo ra relative state factors giữa submap state $\mathbf{x}^i$ và endpoints $\mathbf{x}_L^i$ và $\mathbf{x}_R^i$ tương ứng từ frame đầu tiên và cuối cùng trong submap ($\mathbf{x}_1$ và $\mathbf{x}_{N_{sub}}$) sao cho chúng thỏa mãn các mối quan hệ trạng thái tương đối:

$$e_L^{EP}(\mathbf{T}^i, \mathbf{x}_L^i) = \|\log(\mathbf{T}^i \mathbf{T}'_1 (\mathbf{T}_L^i)^{-1})\|^2 + \|\mathbf{v}'_1 - (\mathbf{R}^i)^T \mathbf{v}_L^i\|^2 + \|\mathbf{b}'_1 - \mathbf{b}_L^i\|^2, \quad (25)$$

$$e_R^{EP}(\mathbf{T}^i, \mathbf{x}_R^i) = \|\log(\mathbf{T}^i \mathbf{T}'_{N_{sub}} (\mathbf{T}_R^i)^{-1})\|^2 + \|\mathbf{v}'_{N_{sub}} - (\mathbf{R}^i)^T \mathbf{v}_R^i\|^2 + \|\mathbf{b}'_{N_{sub}} - \mathbf{b}_R^i\|^2. \quad (26)$$

Sau đó chúng tôi tạo ra một IMU factor giữa $\mathbf{x}_R^i$ và $\mathbf{x}_L^{i+1}$. Theo cách này, một IMU factor chỉ bao phủ một khoảng thời gian nhỏ (khoảng thời gian quét dữ liệu range) và có thể ràng buộc mạnh mẽ các submap poses trong khi tránh mất thông tin vận tốc và bias được ước lượng bởi local mapping module.

Cuối cùng, hàm mục tiêu cho global trajectory optimization được định nghĩa như:

$$g^{GM}(\mathcal{X}^g) = \sum_{(\mathbf{T}^i, \mathbf{T}^j) \in \mathcal{X}^o} e^{PC}(\mathbf{T}^i, \mathbf{T}^j) + \sum_{\mathbf{T}^i \in \mathcal{X}^g} e^{IMU}(\mathbf{x}_R^{i-1}, \mathbf{x}_L^i) + \sum_{\mathbf{T}^i \in \mathcal{X}^g} (e_L^{EP}(\mathbf{T}^i, \mathbf{x}_L^i) + e_R^{EP}(\mathbf{T}^i, \mathbf{x}_R^i)), \quad (27)$$

trong đó $\mathcal{X}^g$ là tập hợp tất cả submap poses, và $\mathcal{X}^o$ là các cặp submaps có overlap.

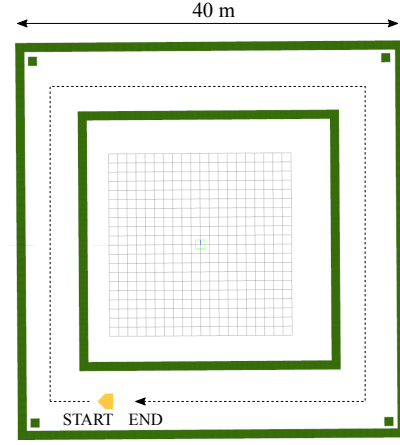
Bất cứ khi nào một submap mới được chèn vào, factor graph được tối ưu hóa incremental sử dụng iSAM2 optimizer [52].

VI. EXPERIMENTS

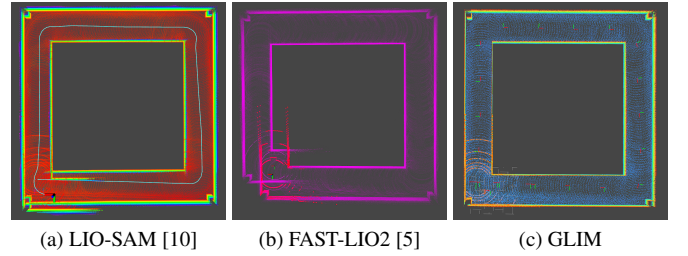
Trong phần này, chúng tôi đầu tiên thể hiện tính mạnh mẽ của thuật toán ước lượng odometry được đề xuất đối với sự suy thoái tạm thời của dữ liệu range trong môi trường mô phỏng và thực tế (Sec. VI-A). Sau đó chúng tôi chứng minh rằng framework được đề xuất có thể được áp dụng cho các cảm biến range-IMU đa dạng (Sec. VI-B). Cuối cùng, chúng tôi so sánh độ chính xác của framework được đề xuất và các phương pháp state-of-the-art và trình bày các nghiên cứu ablation trên Multi-Camera Newer College Dataset [57] (Sec. VI-C) và NTU VIRAL Dataset [58] (Sec. VI-D)².

A. Robustness to Degenerated Range Data

Đánh giá trong mô phỏng: Để thể hiện rằng phương pháp được đề xuất mạnh mẽ đối với sự suy thoái tạm thời của dữ liệu



Hình 8. Môi trường mô phỏng. Môi trường rộng 40 m. Phạm vi quan sát LiDAR bị giới hạn ở 15 m. Dữ liệu LiDAR trở nên hoàn toàn suy thoái ở giữa hành lang.



Hình 9. Kết quả ước lượng cho mức nhiễu IMU thấp nhất (1.0×10^{-3} [m/s²] và [°/s]). Các phương pháp hiện có dựa trên frame-to-model matching nhạy cảm với dữ liệu LiDAR suy thoái trong khi phương pháp được đề xuất tạo ra kết quả ước lượng quỹ đạo tốt.

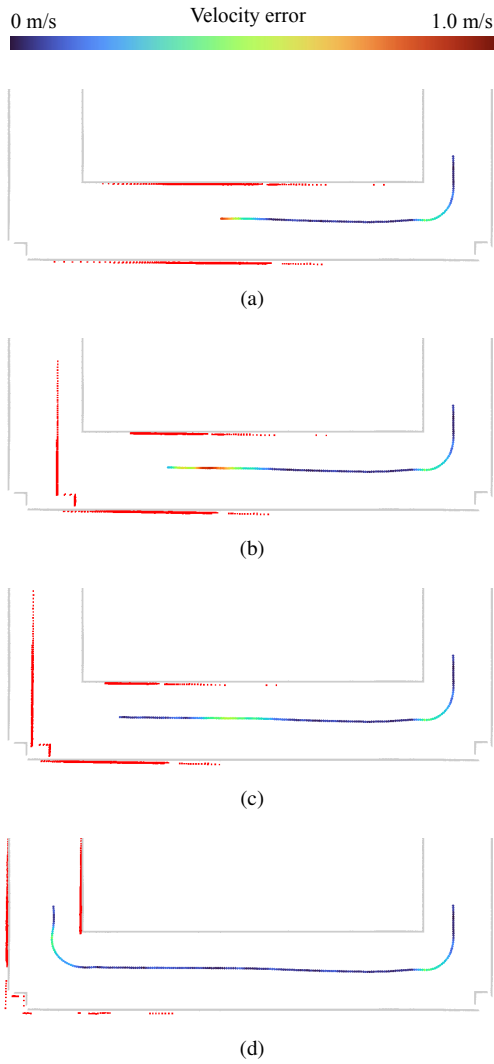
range, chúng tôi đã tiến hành một đánh giá trong môi trường mô phỏng được thể hiện trong Hình 8. Chúng tôi tạo ra point clouds với một model LiDAR (Velodyne VLP-16) trong khi di chuyển nó dọc theo hành lang, và tổng hợp dữ liệu IMU sử dụng OpenVINS [59]. Chúng tôi tạo ra năm chuỗi LiDAR-IMU trong khi thay đổi mức độ nhiễu gia tốc tuyến tính và vận tốc góc IMU ([m/s²] và [°/s]) từ 1.0×10^{-3} đến 1.0×10^{-1} . Mặc dù môi trường rộng 40 m, phạm vi quan sát tối đa của LiDAR bị giới hạn ở 15 m để dữ liệu range trở nên hoàn toàn suy thoái ở giữa hành lang trong vài giây.

Chúng tôi chạy thuật toán ước lượng odometry được đề xuất và hai phương pháp LiDAR-IMU odometry state-of-the-art, cụ thể là LIO-SAM [10], một phương pháp LiDAR-IMU odometry loosely coupled, và FAST-LIO2 [5], một phương pháp LiDAR-IMU odometry tightly coupled với iterated Kalman filter, để đánh giá tính mạnh mẽ của chúng đối với sự suy thoái dữ liệu range. Các quỹ đạo được ước lượng được đánh giá với metric absolute trajectory error (ATE [60]) sử dụng toolkit *evo*³. Lưu ý rằng loop closure bị vô hiệu hóa cho tất cả các phương pháp.

Hình 9 thể hiện các kết quả ước lượng dưới mức nhiễu IMU thấp nhất (1.0×10^{-3}). LIO-SAM và FAST-LIO, dựa trên frame-to-model matching, nhạy cảm với sự suy thoái tạm thời của dữ liệu LiDAR và do đó kết quả ước lượng của chúng có lỗi lớn. Bởi vì frame-to-model matching cần xác định và sửa ngay lập tức sensor pose để hợp nhất point cloud mới nhất vào target

²Các dataset và kết quả ước lượng có sẵn online: <https://staff.aist.go.jp/k.koide/projects/glimsupp>

³<https://michaelgrupp.github.io/evo/>



Hình 10. Sửa lỗi ước lượng vận tốc bởi phương pháp được đề xuất cho cảm biến di chuyển. Màu sắc quỹ đạo mã hóa độ lớn của lỗi ước lượng vận tốc và các điểm đỏ thể hiện LiDAR scan mới nhất. Các điểm bản đồ được thể hiện bằng màu xám để hỗ trợ visualization. Lỗi ước lượng tích lũy ở giữa hành lang do dữ liệu LiDAR suy thoái. Một khi tường đối diện trở nên có thể quan sát được, lỗi ước lượng của frame mới nhất được sửa. Sự sửa chữa này sau đó được lan truyền đến các frame trong quá khứ và một ước lượng chính xác được khôi phục ở cuối.

map model, các phương pháp này không thể tránh sự hỏng hóc của map model khi point clouds bị suy thoái.

Phương pháp được đề xuất có lỗi ước lượng quỹ đạo nhỏ dưới sự suy thoái của dữ liệu LiDAR. Hình 10 thể hiện cách lỗi ước lượng vận tốc cho phương pháp được đề xuất được sửa trong khi cảm biến di chuyển trong hành lang. Màu sắc của quỹ đạo mã hóa độ lớn của lỗi vận tốc và các điểm đỏ thể hiện một lát cắt 2D của point cloud mới nhất. Trong khi lỗi ước lượng vận tốc tích lũy ở giữa hành lang do point clouds bị suy thoái (Hình 10 (a)), một khi các matching cost factors trở nên được ràng buộc tốt bằng cách quan sát tường đối diện, ước lượng của frame mới nhất được sửa (Hình 10 (b)). Sau đó nó được lan truyền đến các frame trong quá khứ (Hình 10 (c)). Các ước lượng vận tốc của tất cả các frame trong quá khứ được sửa và một ước lượng quỹ đạo chính xác được khôi phục ở cuối (Hình 10 (d)).

Bảng I tóm tắt các ATE cho mỗi phương pháp dưới một

Bảng I
RANGE DATA DEGENERATION TEST RESULTS IN SIMULATION

IMU Noise [m/s ²] [°/s]	Absolute Trajectory Error [m]		
	LIO-SAM [10]	FAST-LIO2 [5]	GLIM
1.0×10^{-3}	1.473 ± 0.696	1.294 ± 0.691	0.099 ± 0.012
5.0×10^{-3}	4.670 ± 1.863	1.449 ± 0.843	0.064 ± 0.029
1.0×10^{-2}	12.888 ± 5.674	2.495 ± 1.111	0.133 ± 0.083
5.0×10^{-2}	X	11.735 ± 4.224	0.384 ± 0.163
1.0×10^{-1}	X	X	1.566 ± 0.839

X indicates that the estimation was corrupted.



(a) Environment



(b) Used sensor

Hình 11. Môi trường thí nghiệm và các cảm biến được sử dụng. Hệ thống LiDAR được di chuyển giữa hai cột trong khi đối mặt với tường phẳng để dữ liệu LiDAR trở nên suy thoái ở giữa đường đi. Các fiducial tags và dữ liệu camera được sử dụng để thu được quỹ đạo ground truth.

số mức nhiễu IMU. Ngay cả dưới mức nhiễu IMU thấp nhất (1.0×10^{-3}), LIO-SAM và FAST-LIO2 nhạy cảm với dữ liệu LiDAR suy thoái và thể hiện ATE lớn (1.474 và 1.294 m, tương ứng). Khi nhiễu IMU trở nên lớn hơn, các ATE cho các phương pháp này cũng trở nên lớn hơn. Đầu ra của LIO-SAM bị hỏng khi mức nhiễu được đặt ở 5.0×10^{-2} . Mặc dù FAST-LIO2, sử dụng LiDAR-IMU tight coupling dựa trên iterated-Kalman-filter, mạnh mẽ hơn đối với nhiễu IMU so với LIO-SAM, đầu ra của nó bị hỏng dưới mức nhiễu IMU cao nhất (1.0×10^{-1}).

Phương pháp được đề xuất thể hiện ATE tốt nhất trong số các phương pháp được so sánh dưới tất cả các cài đặt mức nhiễu IMU (0.099 đến 1.566 m) nhờ vào sự kết hợp của fixed-lag smoothing và keyframe-based point cloud matching. Mặc dù lỗi ước lượng của nó tăng dần khi nhiễu IMU trở nên lớn hơn, phương pháp được đề xuất có ATE hợp lý (1.566 m) ngay cả dưới mức nhiễu IMU cao nhất (1.0×10^{-1}).

Đánh giá trong môi trường thực tế: Để thể hiện rằng phương pháp được đề xuất có thể xử lý sự suy thoái dữ liệu range trong tình huống thực tế, chúng tôi đã ghi lại tám chuỗi dữ liệu LiDAR-IMU trong môi trường được thể hiện trong Hình 11 với một LiDAR quét không lặp lại (Livox Avia). LiDAR được di chuyển giữa hai cột trong khi đối mặt với tường phẳng, và do đó dữ liệu LiDAR bị suy thoái ở giữa đường đi cho mỗi chuỗi. Để có được ground truth sensor trajectories, chúng tôi đặt các fiducial tags (AprilTag [62]) trên tường và ước lượng poses của chúng bằng cách sử dụng full bundle adjustment trước. Cho mỗi chuỗi, chúng tôi ghi lại hình ảnh sử dụng camera (OMRON SENTECH STC-MBS202POE) cùng với dữ liệu LiDAR-IMU. Chúng tôi ước lượng camera trajectory thông qua batch optimization của tag projection và IMU constraints như một ground truth trajectory. LiDAR và camera được đồng bộ hóa với host PC thông qua IEEE 1588 protocol (PTP) và

Bảng II
RANGE DATA DEGENERATION TEST RESULTS IN REAL ENVIRONMENT

Seq.	Path length [m]	Duration [s]	Velocity (Avg / Max)		Absolute Trajectory Error [m]		
			Linear [m/s]	Angular [°/s]	FAST-LIO2 [5]	VoxelMap [61]	GLIM
01	4.106	18.5	0.22 / 0.55	11.6 / 39.9	0.815 ± 0.500	0.577 ± 0.157	0.118 ± 0.047
02	4.192	13.6	0.32 / 0.62	13.4 / 33.6	0.822 ± 0.255	0.146 ± 0.057	0.299 ± 0.116
03	4.766	14.4	0.33 / 0.65	18.0 / 43.7	0.873 ± 0.364	0.950 ± 0.263	0.040 ± 0.015
04	4.213	13.1	0.33 / 0.62	14.1 / 51.3	1.137 ± 0.412	0.586 ± 0.299	0.389 ± 0.145
05	4.121	14.0	0.30 / 0.69	17.1 / 50.0	1.048 ± 0.582	0.786 ± 0.400	0.228 ± 0.083
06	3.725	8.8	0.43 / 0.79	16.0 / 42.7	15.551 ± 8.780	0.807 ± 0.309	0.056 ± 0.023
07	3.086	8.9	0.35 / 0.64	17.6 / 53.1	0.635 ± 0.218	0.366 ± 0.111	0.017 ± 0.007
08	2.229	12.1	0.19 / 0.49	14.2 / 31.7	0.297 ± 0.110	0.279 ± 0.082	0.146 ± 0.066

transformation giữa chúng được hiệu chỉnh bằng cách sử dụng normalized-information-distance-based direct LiDAR-camera alignment method [63].

Ngoài LIO-SAM [10] và FAST-LIO2 [5], chúng tôi chạy VoxelMap [61], một ước lượng LiDAR-IMU odometry tightly coupled với adaptive resolutional voxel mapping.

Bảng II tóm tắt các ATE cho phương pháp được đề xuất và FAST-LIO2 và VoxelMap. Lưu ý rằng chúng tôi không thể có được kết quả ước lượng tốt với LIO-SAM [10]. Điều này có thể do LIO-SAM sử dụng cách tiếp cận LiDAR-IMU fusion loosely coupled, trong đó point cloud scan matching được thực hiện riêng biệt mà không có IMU constraints. LIO-SAM do đó gặp khó khăn trong việc xử lý dữ liệu LiDAR bị suy thoái nghiêm trọng. Từ Bảng II, chúng ta có thể thấy rằng FAST-LIO2 tạo ra kết quả ước lượng tốt hơn (0.297 đến 15.551 m). Mặc dù VoxelMap thể hiện kết quả ước lượng tốt hơn so với FAST-LIO2 (0.146 đến 0.950 m) nhờ vào adaptive environment representation và data association, nó cũng chịu ảnh hưởng lớn từ dữ liệu LiDAR bị suy thoái.

Điều này tiết lộ điểm yếu của các phương pháp state-of-the-art dựa trên frame-to-model matching. Như đã thảo luận trong [5], FAST-LIO2 (và các phương pháp khác dựa trên frame-to-model matching) không thể xử lý các tình huống mà point clouds hoàn toàn bị suy thoái (ví dụ: khi LiDAR đối mặt với tường phẳng, bầu trời, hoặc mặt đất hoặc gần các vật thể).

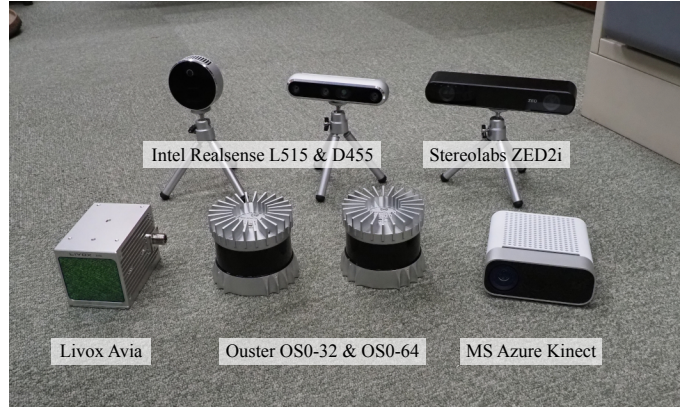
Phương pháp được đề xuất có ATE tốt hơn so với FAST-LIO2 và VoxelMap cho hầu hết các chuỗi (0.017 đến 0.389 m). Chúng tôi giới thiệu người đọc đến trang bổ sung, thể hiện rõ ràng cách tiếp cận fixed-lag smoothing được đề xuất hoạt động và sửa độ trôi ước lượng bằng cách tích cực cập nhật các sensor poses trong quá khứ.

B. Mapping with Various Range-IMU Sensors

Để chứng minh tính linh hoạt của framework được đề xuất, chúng tôi đã tiến hành các thí nghiệm mapping với các cảm biến range-IMU đa dạng với các cơ chế cảm biến khác nhau bao gồm spinning LiDAR (Ouster OS0-32 và OS0-64), non-repetitive scan LiDAR (Livox Avia), time-of-flight (ToF) depth camera (Microsoft Azure Kinect), solid-state LiDAR (Intel Realsense L515), active stereo camera (Intel Realsense D455), và passive stereo camera (Stereolabs ZED2i). Hình 12 thể hiện môi trường thí nghiệm và các cảm biến range-IMU được sử dụng. Tương tự như protocol thu thập ground truth được sử dụng trong Newer College Dataset [64], chúng tôi ước lượng sensor trajectories



(a) Experimental environment



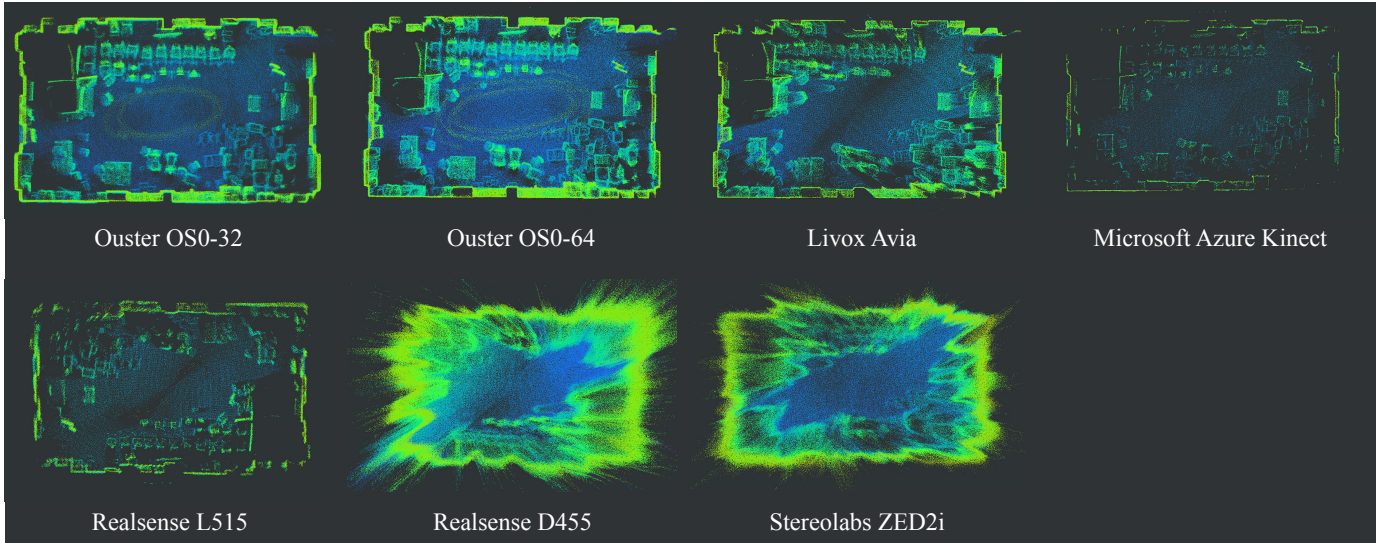
(b) Range-IMU sensors

Hình 12. Môi trường thí nghiệm và các cảm biến range-IMU được sử dụng. Phương pháp được đề xuất được xác thực với các cảm biến range-IMU đa dạng với các cơ chế cảm biến khác nhau.

bằng cách căn chỉnh point clouds với một environment point cloud được ghi lại bằng survey-grade LiDAR (FARO Focus).

Chúng tôi áp dụng phương pháp được đề xuất cho các chuỗi range-IMU được ghi lại và đánh giá các lỗi ước lượng quỹ đạo bằng cách sử dụng các metric ATE và relative trajectory error (RTE) [60]. Chúng tôi đặt độ dài sub-trajectory của đánh giá RTE là 2 m. Để thể hiện tính tổng quát của thuật toán được đề xuất, chúng tôi sử dụng cùng một bộ tham số mapping cho tất cả các cảm biến. Lưu ý rằng chúng tôi cũng đã thử nghiệm FAST-LIO2 [5]; tuy nhiên, chúng tôi không thể có được kết quả hợp lý cho các cảm biến khác ngoài Ouster và Livox LiDARs.

Bảng III tóm tắt các lỗi ước lượng quỹ đạo. Phương pháp



Hình 13. Kết quả mapping thu được với các cảm biến range-IMU đa dạng. Phương pháp được đề xuất đạt được kết quả mapping nhất quán cho tất cả các cảm biến. Cùng một bộ tham số mapping được sử dụng cho tất cả các cảm biến.

Bảng III
CROSS SENSOR EVALUATION

Sensor	ATE [m]	RTE [m]
Ouster OS0-32	0.037 ± 0.018	0.037 ± 0.017
Ouster OS0-64	0.022 ± 0.009	0.025 ± 0.012
Livox Avia	0.041 ± 0.014	0.043 ± 0.019
Microsoft Azure Kinect	0.007 ± 0.003	0.007 ± 0.003
Intel Realsense L515	0.042 ± 0.018	0.045 ± 0.020
Intel Realsense D455	0.206 ± 0.086	0.331 ± 0.161
Stereolabs ZED2i	0.139 ± 0.062	0.194 ± 0.102

ATE: absolute trajectory error; RTE: relative trajectory error

được đề xuất thể hiện kết quả ước lượng chính xác cho các LiDAR và ToF depth camera (OS0-32, OS0-64, Livox Avia, Azure Kinect, Realsense L515), cung cấp dữ liệu point cloud chính xác (ATE: 0.007 đến 0.042 m, RTE: 0.007 đến 0.045 m). Mặc dù kết quả ước lượng suy giảm nhẹ cho các camera dựa trên stereo (Realsense D455 và Stereolabs ZED2i) do point clouds bị biến dạng mạnh, các lỗi ước lượng vẫn ở mức hợp lý (ATE: 0.139 và 0.206 m, RTE: 0.194 và 0.331 m).

Hình 13 thể hiện các kết quả mapping. Cho tất cả các cảm biến, phương pháp được đề xuất đạt được kết quả mapping nhất quán. Mặc dù kết quả mapping cho các cảm biến dựa trên stereo bị biến dạng do biến dạng lớn của input point clouds, chúng tôi có được các bản đồ nhất quán mà không có tương và sần bị nhân đôi.

Những kết quả này thể hiện tính mạnh mẽ của thuật toán ước lượng odometry được đề xuất đối với nhiễu trên point clouds. Các phương pháp ước lượng odometry thông thường dựa trên feature-based point cloud matching nhạy cảm với nhiễu lớn như vậy trong input point clouds, làm cho việc trích xuất edge và plane feature points khó khăn. Bởi vì thuật toán ước lượng odometry được đề xuất dựa trên direct distribution-to-distribution point cloud matching, tránh noise-sensitive feature extraction và sử dụng probabilistic surface modeling, nó có thể xử lý mạnh mẽ các point clouds có nhiễu.

Hơn nữa, những kết quả này chứng minh tính linh hoạt của thuật toán global optimization được đề xuất dựa trên global matching cost minimization. Bởi vì chúng ta không thể mong đợi kết quả frame-by-frame scan matching hợp lý cho các point clouds có nhiễu của các cảm biến dựa trên stereo, các phương pháp hiện có dựa trên pose graph optimization gặp khó khăn trong việc đồng bộ hóa chính xác các vòng lặp. Ngược lại, cách tiếp cận global matching cost minimization không yêu cầu rõ ràng relative pose giữa các frame cho mỗi factor. Bởi vì nó trực tiếp giảm thiểu multi-scan registration errors trên factor graph, nó mạnh mẽ đối với các lỗi frame-by-frame matching như vậy.

C. Quantitative Estimation Accuracy Evaluation on the Newer College Dataset

Chúng tôi đánh giá định lượng độ chính xác ước lượng của framework được đề xuất trên Multi-Camera Newer College Dataset [57], [64]. Dataset này cung cấp các chuỗi dữ liệu LiDAR-camera-IMU được ghi lại với Ouster OS0-128 (10-Hz point clouds và 100-Hz IMU) và Sevensense Alphasense Core (30-Hz images $\times$ 4 hardware-synchronized cameras) trong một số môi trường trong nhà và ngoài trời. LiDAR và cameras được đồng bộ hóa thông qua IEEE 1588 protocol (PTP). Bảng IV tóm tắt thời lượng dữ liệu và độ dài đường đi của mỗi chuỗi trong dataset.

So sánh với state-of-the-art: Chúng tôi so sánh độ chính xác ước lượng quỹ đạo của phương pháp được đề xuất với các phương pháp LiDAR-IMU SLAM state-of-the-art, bao gồm một phương pháp LiDAR-IMU loosely coupled (LIO-SAM [10]), các phương pháp tightly coupled dựa trên iterated Kalman filter (LINS [3], FAST-LIO2 [5]), một phương pháp SLAM liên tục tightly coupled (CLINS [11]), và một phương pháp loosely coupled dựa trên GICP matching (DLO [14]). Bởi vì LINS, FAST-LIO2, và DLO không có cơ chế loop closure, chỉ có lỗi ước lượng odometry được đánh giá cho các phương pháp này. Chúng tôi đánh giá các lỗi ước lượng với các metric ATE và RTE (delta = 10 m).

Bảng IV
DATA DURATIONS AND PATH LENGTHS FOR MULTI-CAMERA NEWER COLLEGE DATASET

Sequence	quad easy	quad medium	quad hard	stairs	park	cloister	math easy	math medium	math hard
Duration [s]	198.7	190.6	187.8	118.9	1572.0	278.6	215.9	176.9	243.7
Path length [m]	246.7	260.4	234.8	57.0	2396.2	428.8	263.6	304.3	320.6

Bảng V
ABSOLUTE TRAJECTORY ERROR [M] RESULTS FOR VARIOUS METHODS ON MULTI-CAMERA NEWER COLLEGE DATASET

Method	Loop closure	quad easy	quad medium	quad hard	stairs	park	cloister	math easy	math medium	math hard	Average
LINS [3]		0.160	0.212	16.824	3.405	0.612	1.170	0.216	0.259	5.710	3.174
LIO-SAM [10]	✓	0.086	0.069	0.105	3.438	1.381	0.085	0.088	0.114	0.089	0.606
FAST-LIO2 [5]		0.068	0.059	0.050	1.320	0.319	0.078	0.091	0.134	0.058	0.454
CLINS [11]	✓	0.197	0.674	0.151	2.336	18.265	0.293	0.225	0.235	0.494	2.541
DLO [14]		0.252	0.451	0.140	2.321	2.757	0.262	0.241	0.242	0.484	0.795
GLIM	✓	0.070	0.061	0.044	0.106	0.459	0.063	0.096	0.119	0.064	0.120
		0.070	0.059	0.043	0.046	0.269	0.056	0.082	0.119	0.048	0.088

Red, blue, and black bold values are respectively the **best**, **second best**, and **third best** results.

Bảng VI
RELATIVE TRAJECTORY ERROR [M] RESULTS FOR VARIOUS METHODS ON MULTI-CAMERA NEWER COLLEGE DATASET

Method	Loop closure	quad easy	quad medium	quad hard	stairs	park	cloister	math easy	math medium	math hard	Average
LINS [3]		0.328	0.463	0.996	4.592	0.435	0.923	0.306	0.523	0.887	1.050
LIO-SAM [10]	✓	0.143	0.172	0.201	2.662	0.174	0.109	0.129	0.223	0.246	0.451
FAST-LIO2 [5]		0.151	0.195	0.209	2.630	0.205	0.117	0.118	0.216	0.211	0.450
CLINS [11]	✓	0.150	0.161	0.152	1.137	0.090	0.103	0.146	0.293	0.102	0.259
DLO [14]		0.177	0.271	0.196	3.070	0.291	0.149	0.145	0.276	0.120	0.522
		0.202	0.486	0.194	2.933	0.157	0.155	0.145	0.282	0.163	0.524
		0.222	0.328	0.365	0.192	0.233	0.240	0.053	0.108	0.379	0.236
GLIM	✓	0.151	0.168	0.164	0.169	0.096	0.100	0.125	0.250	0.137	0.151
		0.154	0.170	0.164	0.096	0.095	0.107	0.123	0.243	0.135	0.143

Red, blue, and black bold values are respectively the **best**, **second best**, and **third best** results.

Bảng VII
PROCESSING TIME FOR *park* SEQUENCE

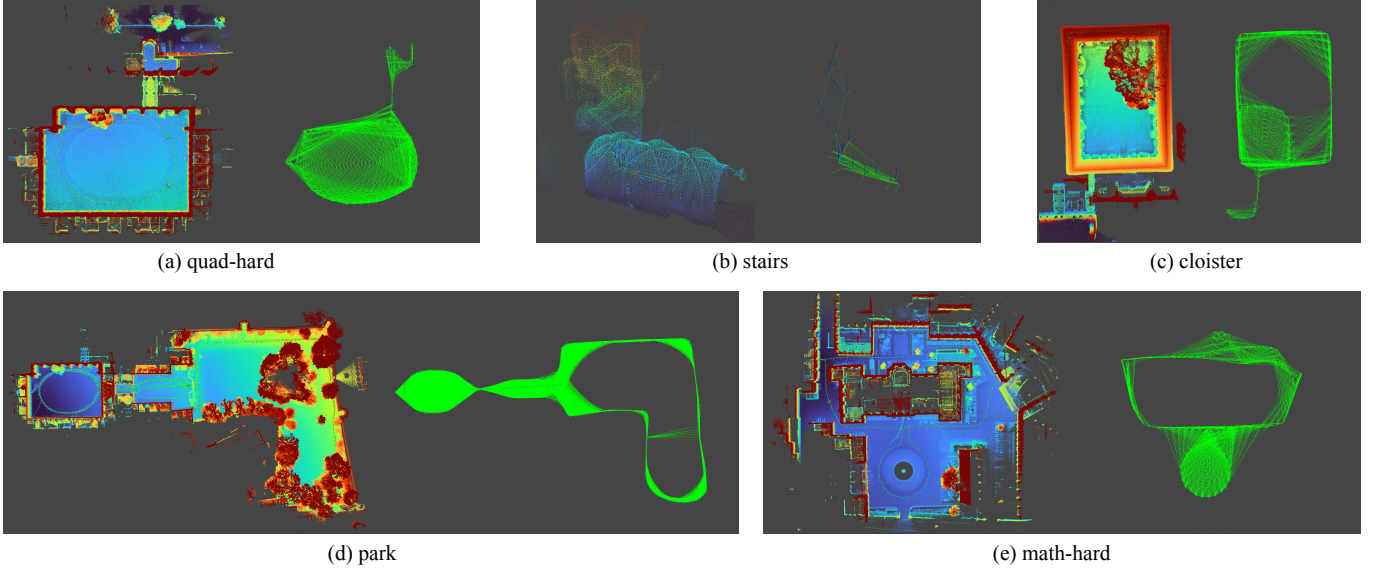
Module	Process	Time [ms]
Preprocess	Downsampling	8.9 ± 1.5
	kNN search	5.7 ± 0.4
	Total (per frame)	15.1 ± 1.6
Odometry estimation	Factor creation	3.3 ± 0.3
	Optimization	25.7 ± 30.5
	Total (per frame)	29.3 ± 30.5
Local mapping	Factor creation	12.3 ± 2.0
	Total (per frame)	12.4 ± 2.0
	Optimization	295.0 ± 111.0
	Total (per submap)	295.0 ± 111.0
Global mapping	Factor creation	20.1 ± 10.3
	Optimization	35.6 ± 51.4
	Total (per submap)	55.8 ± 55.6

Bảng V và VI tương ứng tóm tắt các ATE và RTE cho các phương pháp được đánh giá. Trong số các phương pháp hiện có,

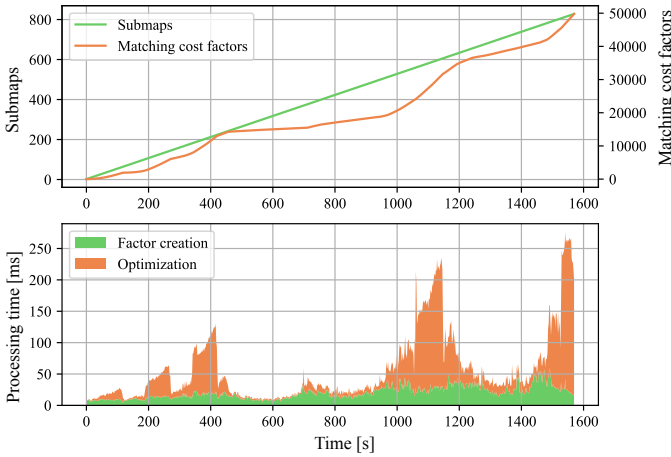
FAST-LIO2 thể hiện ATE và RTE tốt nhất cho hầu hết các chuỗi nhờ vào ước lượng trạng thái mạnh mẽ dựa trên tightly coupled range và IMU data fusion trên iterated Kalman filter. Tuy nhiên, các ATE và RTE cho FAST-LIO2 và các phương pháp khác dựa trên point-to-plane (và point-to-edge) point cloud matching (LINS, LIO-SAM, và CLINS) suy giảm đáng kể cho chuỗi *stairs* trong môi trường trong nhà nhỏ.

Chúng tôi cho rằng điều này là do point-to-plane-based matching yêu cầu point association chính xác và cần điều chỉnh cẩn thận các tham số để xử lý thay đổi môi trường. Đáng chú ý rằng mặc dù chúng tôi fine-tuned các tham số của các phương pháp này để cải thiện kết quả trên chuỗi *stairs*, chúng tôi không thể có được cải thiện mà không hy sinh độ chính xác cho các chuỗi khác trong môi trường ngoài trời. DLO thể hiện ATE và RTE tốt hơn cho chuỗi *stairs* vì nó sử dụng thuật toán direct scan matching dựa trên GICP, không yêu cầu feature extraction và mạnh mẽ đối với thay đổi môi trường. Tuy nhiên, độ chính xác tổng thể của DLO hơi kém hơn FAST-LIO2.

Thuật toán ước lượng odometry được đề xuất thể hiện ATE và RTE tương đương với FAST-LIO2 cho hầu hết các chuỗi. Cho



Hình 14. Kết quả mapping và factor graphs cho Multi-Camera Newer College Dataset. Các bản đồ môi trường có tính nhất quán cao được tạo ra thông qua global matching cost minimization trên các factor graphs cực kỳ dày đặc.



Hình 15. Số lượng submaps và matching cost factors và thời gian xử lý của module global optimization cho chuỗi *park*. Nhờ vào scheme tối ưu hóa hiệu quả và incremental, phương pháp được đề xuất đạt được xử lý thời gian thực.

chuỗi *stairs*, nó thể hiện kết quả ước lượng tốt nhất trong số tất cả các phương pháp được đánh giá. Điều này cho thấy rằng phương pháp được đề xuất có thể xử lý nhiều loại môi trường khác nhau trong khi ước chế độ trôi ước lượng. Với loop closure, các ATE và RTE cho phương pháp được đề xuất được cải thiện thêm, và chúng tôi đạt được ATE tốt nhất cho hầu hết các chuỗi. Hình 14 thể hiện map point clouds và factor graphs được tạo ra với framework được đề xuất. Chúng ta có thể thấy rằng phương pháp được đề xuất tạo ra một factor graph cực kỳ dày đặc cho global matching cost minimization, dẫn đến kết quả mapping nhất quán.

Processing time: Bảng VII tóm tắt thời gian xử lý, được đo với Intel Core i7 8700K và NVIDIA RTX 1660 Ti, cho mỗi module trong framework được đề xuất chạy qua chuỗi *park*, là chuỗi dài nhất trong dataset.

Các module preprocessing và odometry estimation lần lượt mất 15.1 và 29.3 ms mỗi frame trung bình và do đó đủ nhanh

hơn yêu cầu thời gian thực (100 ms mỗi frame). Submap optimization, được thực hiện khoảng mỗi 2 s, mất 295.0 ms trung bình. Global map optimization mất 55.8 ms trung bình cho mỗi lần tạo submap. Hình 15 thể hiện cách thời gian global optimization tăng khi số lượng submaps và matching cost factors tăng. Mặc dù một số lượng lớn matching cost factors được tạo ra (hơn 50,000 factors giữa 800 submaps), global optimization hội tụ trong ít hơn 50 ms cho hầu hết các frame nhờ vào GPU-accelerated matching cost factor và iSAM2, cập nhật hiệu quả chỉ các sub Bayes trees có thể bị ảnh hưởng bởi việc chèn một submap mới. Mặc dù global optimization mất thời gian dài hơn khi đóng một vòng lặp lớn, thời gian tối ưu hóa tối đa chỉ khoảng 250 ms, nhỏ hơn nhiều so với yêu cầu thời gian thực (2 s). Lưu ý rằng linearization của matching cost factors chiếm phần lớn thời gian tối ưu hóa; linear solver thực hiện trên CPU chỉ chiếm khoảng 5% tổng thời gian tối ưu hóa.

Odometry estimation ablation study: Để kiểm tra tác động của surface-orientation-based correspondence validation và multi-resolution voxelmap, chúng tôi chạy framework được đề xuất sử dụng cùng cài đặt như trong thí nghiệm trước nhưng không có các chức năng này. Chúng tôi cũng chạy thuật toán ước lượng odometry được đề xuất với multi-camera constraints để thể hiện rằng chúng tôi có thể cải thiện độ chính xác mapping trong tình huống thực tế bằng cách giới thiệu các ràng buộc bổ sung thông qua global callback slot mechanism. Bảng VIII tóm tắt các kết quả đánh giá.

Không có surface-orientation-based correspondence validation, các ATE suy giảm cho khoảng một nửa các chuỗi (*stairs*, *park*, *math-easy*, *math-medium*, và *math-hard*) và ATE trung bình tăng từ 0.120 lên 0.140 m. Chúng tôi cho rằng điều này là do các điểm trên một bề mặt có thể bị liên kết sai với các điểm ở phía bên kia của bề mặt trong frame khác khi correspondence validation không được thực hiện.

Không có multi-resolution voxelmap, chúng tôi quan sát thấy cải thiện độ chính xác cho chuỗi *stairs* (ATE giảm từ 0.106 xuống 0.073 m). Tuy nhiên, cho tất cả các chuỗi khác, các ATE

Bảng VIII
ABSOLUTE TRAJECTORY ERROR [M] RESULTS FOR ODOMETRY ESTIMATION ABLATION STUDY

Method	quad easy	quad medium	quad hard	stairs	park	cloister	math easy	math medium	math hard	Average
GLIM odometry (baseline)	0.070	0.061	0.044	0.106	0.459	0.063	0.096	0.119	0.064	0.120
w/o Surface validation	0.070 (+0.000)	0.061 (+0.000)	0.043 (-0.001)	0.114 (+0.008)	0.533 (+0.074)	0.061 (-0.002)	0.140 (+0.044)	0.147 (+0.027)	0.097 (+0.033)	0.140 (+0.020)
w/o Multi resolution	0.078 (+0.008)	0.092 (+0.031)	0.071 (+0.027)	0.073 (-0.033)	1.096 (+0.637)	0.355 (+0.292)	0.127 (+0.031)	0.140 (+0.021)	0.074 (+0.010)	0.234 (+0.114)
w/ Multi-cam constraints	0.069 (-0.001)	0.059 (-0.002)	0.040 (-0.004)	0.045 (-0.061)	0.418 (-0.041)	0.053 (-0.010)	0.095 (-0.001)	0.122 (+0.003)	0.055 (-0.009)	0.106 (-0.014)

Red and blue values are respectively **improved** and **deteriorated** results.
Values in parentheses indicate absolute trajectory error differences from those of the baseline.

Bảng IX
ABSOLUTE TRAJECTORY ERROR [M] RESULTS FOR GLOBAL OPTIMIZATION ABLATION STUDY

Method	quad easy	quad medium	quad hard	stairs	park	cloister	math easy	math medium	math hard	Average
GLIM (Baseline)	0.070	0.059	0.043	0.046	0.269	0.056	0.082	0.119	0.048	0.088
w/o IMU factors	0.070 (+0.000)	0.059 (+0.000)	0.045 (+0.002)	0.054 (+0.008)	0.268 (-0.001)	0.217 (+0.161)	0.081 (-0.001)	0.119 (+0.000)	0.048 (+0.000)	0.107 (+0.019)
w/ IMU factors between submaps	0.070 (+0.000)	0.059 (+0.000)	0.043 (+0.000)	0.054 (+0.008)	0.268 (-0.001)	0.341 (+0.285)	0.082 (+0.000)	0.119 (+0.000)	0.048 (+0.000)	0.120 (+0.032)

Red and blue values are respectively **improved** and **deteriorated** results.
Values in parentheses indicate absolute trajectory error differences from those of the baseline.

suy giảm đáng kể, dẫn đến ATE trung bình suy giảm (0.234 m). Các ATE có xu hướng lớn cho các môi trường ngoài trời lớn (ví dụ: *park* và *cloister*). Kết quả chỉ ra rằng multi-resolution voxelmap cho phép liên kết mạnh mẽ giữa points và voxels trong nhiều loại môi trường khác nhau với chi phí giảm nhẹ độ chính xác trong môi trường trong nhà nhỏ.

Với multi-camera visual constraints, các ATE cải thiện cho tất cả các chuỗi ngoại trừ *math-medium* và ATE trung bình giảm từ 0.120 xuống 0.106 m. Cải thiện ATE là nhỏ trong các chuỗi mà có nhiều đặc trưng hình học và scan matching được ràng buộc tốt. Các ATE cho các chuỗi *stairs*, *park*, *cloister*, và *math-hard*, nơi point cloud matching có thể không ổn định do thay đổi tỷ lệ môi trường và chuyển động cảm biến mạnh, được cải thiện đáng kể với tight coupling của visual constraints.

Global trajectory optimization ablation study: Chúng tôi đánh giá tác động của cơ chế *endpoint* được đề xuất để ổn định hóa submap pose optimization với IMU constraints. Chúng tôi chạy framework được đề xuất với hai cài đặt: 1) không có IMU constraints và 2) với IMU constraints được tạo trực tiếp giữa các trạng thái submap. Bảng IX tóm tắt các kết quả đánh giá. Chúng ta có thể thấy rằng cho cả hai cài đặt, các ATE suy giảm đáng kể cho chuỗi *cloister*. Khi framework chạy trên chuỗi *cloister*, optimizer có lỗi hệ thống tuyến tính không xác định, chỉ ra rằng factor graph bị under-constrained. Ngay cả khi chúng tôi chèn IMU factors trực tiếp giữa các trạng thái submap, optimizer vẫn chịu ảnh hưởng từ hệ thống under-constrained. Điều này là do submaps được tạo ra với khoảng thời gian lớn và IMU factors được tạo giữa chúng có thời gian tích hợp lớn, dẫn đến độ không chắc chắn lớn. Điều này thể hiện ưu điểm của cơ chế *endpoint* được đề xuất, có thể ràng buộc mạnh mẽ các submap poses vì mỗi IMU factor chỉ bao phủ một khoảng quét tối thiểu bằng cách được tạo giữa các *endpoints* của các submaps liên tiếp.

D. Quantitative Estimation Accuracy Evaluation on the NTU VIRAL Dataset

Chúng tôi so sánh thêm độ chính xác của phương pháp được đề xuất với các phương pháp state-of-the-art trong môi trường thách thức hơn trên NTU VIRAL dataset [58]. Dataset này cung cấp các chuỗi đồng bộ thời gian của hai LiDAR (Ouster OS1-16), hai camera (uEye 1221 LE), một IMU (VectorNav VN100), và một UWB (Humatic P440) trên UAV. Bởi vì nó được ghi lại với chuyển động 3D động của UAV bay, chúng tôi cho rằng đánh giá trên dataset này sẽ tiết lộ tính mạnh mẽ của các phương pháp state-of-the-art trong các tình huống thách thức.

Chúng tôi chạy phương pháp được đề xuất với ba cấu hình (LiDAR-IMU, visual-LiDAR-IMU, LiDAR-IMU với loop closure) và so sánh độ chính xác của nó với các phương pháp ước lượng odometry LiDAR-IMU state-of-the-art (LIO-SAM [10], MLOAM [65], FAST-LIO2 [5], VoxelMap [61], và BALM [39]), các phương pháp ước lượng odometry visual-LiDAR-IMU (VIRAL-SLAM [66] và FAST-LIVO [67]), và một phương pháp mapping LiDAR-IMU với loop closure (SLICT [68]). Chúng tôi đánh giá các ATE của các phương pháp này bằng cách sử dụng mã đánh giá được cung cấp bởi dataset⁴.

Bảng X tóm tắt các ATE cho các phương pháp được đánh giá. Các ATE cho LIO-SAM, MLOAM, VIRAL-SLAM, và SLICT được lấy từ [66] và [68].

Trong số các phương pháp dựa trên LiDAR-IMU hiện có, FAST-LIO2 thể hiện kết quả ước lượng chính xác (ATE trung bình: 0.040 m) nhờ vào việc fusion hiệu quả của LiDAR và IMU constraints. Chúng ta cũng có thể thấy rằng BALM cũng thể hiện kết quả tương đương (0.039 m) nhờ vào tối ưu hóa

⁴https://github.com/ntu-aris/viral_eval

Bảng X
RELATIVE TRAJECTORY ERROR [M] RESULTS FOR VARIOUS METHODS ON NTU VIRAL DATASET

Method	Camera	Loop closure	eee 01	eee 02	eee 03	nya 01	nya 02	nya 03	sbs 01	sbs 02	sbs 03	Average
LiDAR-based methods												
LIO-SAM [10]			0.075	0.069	0.101	0.076	0.090	0.137	0.089	0.083	0.140	0.096
MLOAM [65] †			0.249	0.166	0.232	0.123	0.191	0.226	0.173	0.147	0.153	0.184
FAST-LIO2 [25]			0.056	0.031	0.047	0.041	0.047	0.036	0.036	0.033	0.034	0.040
VoxelMap [61]			0.075	0.033	0.070	0.048	0.040	0.050	0.041	0.072	0.058	0.054
BALM [39]			0.060	0.021	0.028	0.031	0.037	0.028	0.041	0.054	0.055	0.039
GLIM			0.034	0.030	0.032	0.031	0.028	0.027	0.037	0.025	0.034	0.031
Visual-LiDAR-based methods												
VIRAL-SLAM [66] †	✓		0.060	0.058	0.037	0.051	0.043	0.032	0.048	0.062	0.054	0.049
FAST-LIVO [67]	✓		0.077	0.020	0.032	0.034	0.044	0.025	0.039	0.044	0.039	0.039
GLIM	✓		0.032	0.026	0.030	0.031	0.028	0.026	0.037	0.025	0.033	0.030
LiDAR-based methods with loop closure												
SLICT [68]		✓	0.032	0.025	0.028	0.023	0.023	0.026	0.030	0.029	0.034	0.028
GLIM		✓	0.025	0.018	0.023	0.025	0.027	0.025	0.031	0.024	0.027	0.025

Black bold values are the best results for each configuration.

† Both the horizontal and vertical LiDARs are used.

dựa trên BA đảm bảo kết quả mapping nhất quán. Phương pháp được đề xuất thể hiện ATE tốt nhất cho sáu trong số chín chuỗi dẫn đến ATE trung bình tốt nhất (0.031 m) trong số các phương pháp dựa trên LiDAR-IMU. Chúng tôi cho rằng tối ưu hóa dựa trên fixed-lag-smoothing cho phép xử lý chuyển động động của UAV một cách mạnh mẽ và đạt được kết quả ước lượng quỹ đạo mượt mà và chính xác.

Với visual constraints, phương pháp được đề xuất cải thiện ATE trung bình (0.030 m) là kết quả tốt nhất trong số các phương pháp dựa trên visual-LiDAR-IMU được đánh giá. Kết quả này xác nhận rằng tight fusion của visual constraints được đề xuất cho phép tăng cường độ chính xác ước lượng mà không hy sinh độ chính xác của ước lượng dựa trên LiDAR-IMU.

Cuối cùng, với loop closure, ATE của phương pháp được đề xuất được cải thiện thêm lên 0.025 m tốt hơn ATE của SLICT (0.028 m). Chúng tôi quan sát thấy cải thiện độ chính xác lớn trong các chuỗi ngoài trời lớn thể hiện hiệu quả của global trajectory optimization được đề xuất để bù đắp cho độ trôi ước lượng odometry trong các tình huống thách thức.

VII. DISCUSSION AND FUTURE WORK

Mặc dù chúng tôi đã thể hiện tính mạnh mẽ của framework được đề xuất đối với sự suy thoái tạm thời của dữ liệu range, vẫn còn thách thức cho range-IMU odometry estimation được đề xuất để xử lý sự suy thoái dữ liệu range dài hạn vì chúng tôi cần giới hạn thời lượng của cửa sổ tối ưu hóa của odometry estimation cho hiệu suất thời gian thực. Có hai cách tiếp cận có thể để làm cho hệ thống mạnh mẽ đối với sự suy thoái dài hạn. Cách tiếp cận đầu tiên là tích hợp một nguồn dữ liệu bổ sung (ví dụ: camera, radar, hoặc wheel odometry) để ước chế độ trôi ước lượng trong quá trình suy thoái của một cảm biến. Cách tiếp cận này đã được áp dụng thành công cho các phương pháp visual-range-IMU [67]. Cách tiếp cận thứ hai là tích hợp motion estimation dựa trên học máy vào odometry estimation. Các phương pháp pedestrian dead reckoning gần đây [69], [70] thể hiện rằng chúng ta có thể giảm hiệu quả độ trôi ước lượng

quỹ đạo chỉ sử dụng dữ liệu IMU với một mô hình motion estimation dựa trên học máy. Các mô hình này cho phép tiềm prior knowledge của motion pattern để giảm độ không chắc chắn của ước lượng vận tốc dựa trên IMU trong quá trình suy thoái dữ liệu range. Chúng tôi tin rằng tính mở rộng của GLIM làm cho việc triển khai và đánh giá cả hai cách tiếp cận để xử lý sự suy thoái dữ liệu range dài hạn trở nên dễ dàng.

Về global trajectory optimization, các kết quả thí nghiệm thể hiện rằng thuật toán global optimization được tăng tốc GPU được đề xuất có thể xử lý đủ các tình huống thực tế. Tuy nhiên, cho các vấn đề mapping quy mô lớn hơn (ví dụ: mapping cấp thành phố hoặc quốc gia), một thuật toán tối ưu hóa có khả năng mở rộng hơn là cần thiết. Các thuật toán tối ưu hóa phân tán và phù hợp với GPU như Gaussian belief propagation [71] là các cách tiếp cận hứa hẹn để xử lý các vấn đề cực kỳ lớn. Registration error factor dựa trên GPU được đề xuất sẽ phù hợp tốt với các thuật toán tối ưu hóa dựa trên GPU như vậy.

VIII. CONCLUSION

Bài báo này trình bày GLIM, một framework SLAM range-IMU 3D với tăng tốc GPU. Bằng cách tận dụng hoàn toàn sức mạnh tính toán của GPU hiện đại, chúng tôi đề xuất các thuật toán ước lượng odometry và tối ưu hóa quỹ đạo toàn cục cho phép định vị và lập bản đồ mạnh mẽ và chính xác trong các tình huống thách thức. Các kết quả thí nghiệm thể hiện rằng các phương pháp state-of-the-art dựa trên frame-to-model matching gặp khó khăn trong việc xử lý dữ liệu range hoàn toàn suy thoái trong khi phương pháp được đề xuất có thể ước lượng quỹ đạo cảm biến một cách mạnh mẽ trong các tình huống như vậy.

TÀI LIỆU

- [1] J. Zhang and S. Singh, "Low-drift and real-time lidar odometry and mapping," *Autonomous Robots*, vol. 41, no. 2, pp. 401–416, Feb. 2016.
- [2] H. Ye, Y. Chen, and M. Liu, "Tightly coupled 3D lidar inertial odometry and mapping," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2019, pp. 3144–3150.

- [3] C. Qin, H. Ye, C. E. Pranata, J. Han, S. Zhang, and M. Liu, "LINS: A lidar-inertial state estimator for robust and efficient navigation," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2020, pp. 8899–8906.
- [4] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-LIO: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4861–4868, Apr. 2022.
- [5] W. Xu, Y. Cai, D. He, J. Lin, and F. Zhang, "FAST-LIO2: Fast direct LiDAR-inertial odometry," *IEEE Transactions on Robotics*, pp. 1–21, Jan. 2022.
- [6] G. Grisetti, R. Kummerle, C. Stachniss, and W. Burgard, "A tutorial on graph-based SLAM," *IEEE Intelligent Transportation Systems Magazine*, vol. 2, no. 4, pp. 31–43, Dec. 2010.
- [7] D. Landry, F. Pomerleau, and P. Giguere, "CELLO-3D: Estimating the covariance of ICP in the real world," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2019, pp. 8190–8196.
- [8] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Globally consistent and tightly coupled 3d LiDAR inertial mapping," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2022, pp. 5622–5628.
- [9] T. Shan and B. Englot, "LeGO-LOAM: Lightweight and ground-optimized lidar odometry and mapping on variable terrain," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2018, pp. 4758–4765.
- [10] T. Shan, B. Englot, D. Meyers, W. Wang, C. Ratti, and R. Daniella, "LIO-SAM: Tightly-coupled lidar inertial odometry via smoothing and mapping," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2020, pp. 5135–5142.
- [11] J. Lv, K. Hu, J. Xu, Y. Liu, X. Ma, and X. Zuo, "CLINS: Continuous-time trajectory estimation for lidar-inertial system," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Dec. 2021, pp. 6657–6663.
- [12] J. Behley and C. Stachniss, "Efficient surfel-based SLAM using 3D laser range data in urban environments," in *Robotics: Science and Systems XIV*. Robotics: Science and Systems Foundation, June 2018.
- [13] J. Wang, M. Xu, F. Foroughi, D. Dai, and Z. Chen, "FasterGICP: Acceptance-rejection sampling based 3d lidar odometry," *IEEE Robotics and Automation Letters*, vol. 7, no. 1, pp. 255–262, Jan. 2022.
- [14] K. Chen, B. T. Lopez, A. akbar Agha-mohammadi, and A. Mehta, "Direct LiDAR odometry: Fast localization with dense point clouds," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2000–2007, Apr. 2022.
- [15] Z. Wang, L. Zhang, Y. Shen, and Y. Zhou, "D-LIOM: Tightly-coupled direct LiDAR-inertial odometry and mapping," *IEEE Transactions on Multimedia*, pp. 1–1, Apr. 2022.
- [16] A. Segal, D. Haehnel, and S. Thrun, "Generalized-ICP," in *Robotics: Science and Systems V*. Robotics: Science and Systems Foundation, June 2009, pp. 435–443.
- [17] T. Qin, P. Li, and S. Shen, "VINS-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE Transactions on Robotics*, vol. 34, no. 4, pp. 1004–1020, Aug. 2018.
- [18] L. V. Stumberg, V. Usenko, and D. Cremers, "Direct sparse visual-inertial odometry using dynamic marginalization," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2018, pp. 2510–2517.
- [19] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, and J. D. Tardos, "ORB-SLAM3: An accurate open-source library for visual, visual-inertial, and multimap SLAM," *IEEE Transactions on Robotics*, pp. 1–17, May 2021.
- [20] S. Weiss and R. Siegwart, "Real-time metric state estimation for modular vision-inertial systems," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2011, pp. 4531–4537.
- [21] V. Indelman, S. Williams, M. Kaess, and F. Dellaert, "Information fusion in navigation systems via factor graph based incremental smoothing," *Robotics and Autonomous Systems*, vol. 61, no. 8, pp. 721–738, Aug. 2013.
- [22] S. Zhao, H. Zhang, P. Wang, L. Nogueira, and S. Scherer, "Super odometry: IMU-centric LiDAR-visual-inertial estimator for challenging environments," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Sept. 2021, pp. 8729–8736.
- [23] M. Palieri, B. Morrell, A. Thakur, K. Ebadi, J. Nash, A. Chatterjee, C. Kanellakis, L. Carlone, C. Guaragnella, and A. akbar Agha-mohammadi, "LOCUS: A multi-sensor lidar-centric solution for high-precision odometry and 3d mapping in real-time," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 421–428, Apr. 2021.
- [24] A. Reinke, M. Palieri, B. Morrell, Y. Chang, K. Ebadi, L. Carlone, and A.-A. Agha-Mohammadi, "LOCUS 2.0: Robust and computationally efficient lidar odometry for real-time 3d mapping," *IEEE Robotics and Automation Letters*, pp. 1–8, June 2022.
- [25] W. Xu and F. Zhang, "FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated kalman filter," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3317–3324, Apr. 2021.
- [26] K. Li, M. Li, and U. D. Hanebeck, "Towards high-performance solid-state-LiDAR-inertial odometry and mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5167–5174, July 2021.
- [27] H. Strasdat, J. Montiel, and A. J. Davison, "Visual SLAM: Why filter?" *Image and Vision Computing*, vol. 30, no. 2, pp. 65–77, Feb. 2012.
- [28] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "On-manifold preintegration for real-time visual-inertial odometry," *IEEE Transactions on Robotics*, vol. 33, no. 1, pp. 1–21, Feb. 2017.
- [29] P. Dellenbach, J.-E. Deschaud, B. Jacquet, and F. Goulette, "CT-ICP: Real-time elastic LiDAR odometry with loop closure," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2022.
- [30] D. Droschel and S. Behnke, "Efficient continuous-time SLAM for 3d lidar-based online mapping," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2018.
- [31] C. L. Gentil, T. Vidal-Calleja, and S. Huang, "IN2LAAMA: Inertial lidar localization autocalibration and mapping," *IEEE Transactions on Robotics*, vol. 37, no. 1, pp. 275–290, Feb. 2021.
- [32] G. Cioffi, T. Cieslewski, and D. Scaramuzza, "Continuous-time vs. discrete-time vision-based SLAM: A comparative study," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 2399–2406, Apr. 2022.
- [33] A. Censi, "An accurate closed-form estimate of ICP's covariance," in *IEEE International Conference on Robotics and Automation*. IEEE, Apr. 2007, pp. 3167–3172.
- [34] T. M. Iversen, A. G. Buch, and D. Kraft, "Prediction of ICP pose uncertainties using monte carlo simulation with synthetic depth images," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Sept. 2017, pp. 4640–4647.
- [35] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LiDAR SLAM," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2016, pp. 1271–1278.
- [36] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Globally consistent 3D LiDAR mapping with GPU-accelerated GICP matching cost factors," *IEEE Robotics and Automation Letters*, pp. 1–8, Sept. 2021.
- [37] L. Zhou, S. Wang, and M. Kaess, "π-LSAM: LiDAR smoothing and mapping with planes," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2021, pp. 5751–5757.
- [38] D. Wisth, M. Camurri, and M. Fallon, "VILENS: Visual, inertial, lidar, and leg odometry for all-terrain legged robots," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 309–326, Feb. 2023.
- [39] Z. Liu and F. Zhang, "BALM: Bundle adjustment for lidar mapping," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 3184–3191, Apr. 2021.
- [40] H. Huang, Y. Sun, J. Wu, J. Jiao, X. Hu, L. Zheng, L. Wang, and M. Liu, "On bundle adjustment for multiview point cloud registration," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 8269–8276, Oct. 2021.
- [41] X. Liu, Z. Liu, F. Kong, and F. Zhang, "Large-scale LiDAR consistent mapping using hierarchical LiDAR bundle adjustment," *IEEE Robotics and Automation Letters*, vol. 8, no. 3, pp. 1523–1530, Mar. 2023.
- [42] F. Lu and E. Milios, "Globally consistent range scan alignment for environment mapping," *Autonomous Robots*, vol. 4, no. 4, pp. 333–349, Oct. 1997.
- [43] D. Borrmann, J. Elseberg, K. Lingemann, A. Nüchter, and J. Hertzberg, "Globally consistent 3D mapping with scan matching," *Robotics and Autonomous Systems*, vol. 56, no. 2, pp. 130–142, Feb. 2008.
- [44] J. Sprickerhof, A. Nüchter, K. Lingemann, and J. Hertzberg, "A heuristic loop closing technique for large-scale 6D SLAM," *Automatika*, vol. 52, no. 3, pp. 199–222, Jan. 2011.
- [45] V. Reijgwart, A. Millane, H. Oleynikova, R. Siegwart, C. Cadena, and J. Nieto, "Voxgraph: Globally consistent, volumetric mapping using signed distance function submaps," *IEEE Robotics and Automation Letters*, vol. 5, no. 1, pp. 227–234, Jan. 2020.
- [46] R. Mur-Artal and J. D. Tardos, "Visual-Inertial monocular SLAM with map reuse," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 796–803, Apr. 2017.
- [47] T. Schneider, M. Dymczyk, M. Fehr, K. Egger, S. Lynen, I. Gilitschenski, and R. Siegwart, "Maplab: An open framework for research in visual-inertial mapping and localization," *IEEE Robotics and Automation Letters*, vol. 3, no. 3, pp. 1418–1425, July 2018.
- [48] V. Usenko, N. Demmel, D. Schubert, J. Stuckler, and D. Cremers, "Visual-inertial mapping with non-linear factor recovery," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 422–429, Apr. 2020.

- [49] K. Koide, M. Yokozuka, S. Oishi, and A. Banno, "Voxelized GICP for fast and accurate 3D point cloud registration," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2021, pp. 11 054–11 059.
- [50] M. Nießner, M. Zollhöfer, S. Izadi, and M. Stamminger, "Real-time 3d reconstruction at scale using voxel hashing," *ACM Transactions on Graphics*, vol. 32, no. 6, pp. 1–11, Nov. 2013.
- [51] P. Biber and W. Strasser, "The normal distributions transform: a new approach to laser scan matching," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2003, pp. 2743–2748.
- [52] M. Kaess, H. Johannsson, R. Roberts, V. Ila, J. J. Leonard, and F. Dellaert, "iSAM2: Incremental smoothing and mapping using the bayes tree," *International Journal of Robotics Research*, vol. 31, no. 2, pp. 216–235, Dec. 2011.
- [53] F. Dellaert and G. Contributors, "borglab/gtsam," May 2022. [Online]. Available: <https://github.com/borglab/gtsam>
- [54] J. Engel, V. Koltun, and D. Cremers, "Direct sparse odometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 3, pp. 611–625, Mar. 2018.
- [55] M. Yokozuka, S. Oishi, S. Thompson, and A. Banno, "VITAMIN-e: VISual tracking and MappINg with extremely dense feature points," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, June 2019, pp. 9633–9642.
- [56] K. Levenberg, "A method for the solution of certain non-linear problems in least squares," *Quarterly of Applied Mathematics*, vol. 2, no. 2, pp. 164–168, July 1944.
- [57] L. Zhang, M. Camurri, D. Wisth, and M. Fallon, "Multi-camera lidar inertial extension to the newer college dataset," arXiv:2112.08854, 2021.
- [58] T.-M. Nguyen, S. Yuan, M. Cao, Y. Lyu, T. H. Nguyen, and L. Xie, "NTU VIRAL: A visual-inertial-ranging-lidar dataset, from an aerial vehicle viewpoint," *The International Journal of Robotics Research*, vol. 41, no. 3, pp. 270–280, Nov. 2021.
- [59] P. Geneva, K. Eickenhoff, W. Lee, Y. Yang, and G. Huang, "OpenVINS: A research platform for visual-inertial estimation," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2020, pp. 4666–4672.
- [60] Z. Zhang and D. Scaramuzza, "A tutorial on quantitative trajectory evaluation for visual(-inertial) odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2018, pp. 7244–7251.
- [61] C. Yuan, W. Xu, X. Liu, X. Hong, and F. Zhang, "Efficient and probabilistic adaptive voxel mapping for accurate online LiDAR odometry," *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 8518–8525, July 2022.
- [62] J. Wang and E. Olson, "AprilTag 2: Efficient and robust fiducial detection," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2016, pp. 4193–4198.
- [63] K. Koide, S. Oishi, M. Yokozuka, and A. Banno, "General, single-shot, target-less, and automatic lidar-camera extrinsic calibration toolbox," in *IEEE International Conference on Robotics and Automation*. IEEE, 2023.
- [64] M. Ramezani, Y. Wang, M. Camurri, D. Wisth, M. Mattamala, and M. Fallon, "The newer college dataset: Handheld LiDAR, inertial and vision with ground truth," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, Oct. 2020, pp. 4353–4360.
- [65] J. Jiao, H. Ye, Y. Zhu, and M. Liu, "Robust odometry and mapping for multi-LiDAR systems with online extrinsic calibration," *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 351–371, Feb. 2022.
- [66] T.-M. Nguyen, S. Yuan, M. Cao, T. H. Nguyen, and L. Xie, "Viral slam: Tightly coupled camera-imu-uwblidar slam," May 2021.
- [67] C. Zheng, Q. Zhu, W. Xu, X. Liu, Q. Guo, and F. Zhang, "FAST-LIVO: Fast and tightly-coupled sparse-direct LiDAR-inertial-visual odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, Oct. 2022.
- [68] T.-M. Nguyen, D. Duberg, P. Jensfelt, S. Yuan, and L. Xie, "Sliot: Multi-input multi-scale surfel-based lidar-inertial continuous-time odometry and mapping," Nov.
- [69] S. Herath, H. Yan, and Y. Furukawa, "RoNIN: Robust neural inertial navigation in the wild: Benchmark, evaluations, and new methods," in *IEEE International Conference on Robotics and Automation*. IEEE, May 2020.
- [70] Q. Wang, H. Luo, J. Wang, L. Sun, Z. Ma, C. Zhang, M. Fu, and F. Zhao, "Recent advances in pedestrian navigation activity recognition: A review," *IEEE Sensors Journal*, vol. 22, no. 8, pp. 7499–7518, Apr. 2022.
- [71] A. J. Davison and J. Ortiz, "FutureMapping 2: Gaussian belief propagation for spatial AI," 2019.